

Bachelorarbeit

Pseudo Transient Continuation zur Bestimmung von Gleichgewichtspunkten

Neil Vetter

21. November 2019

Mathematisches Institut
Mathematisch-Naturwissenschaftliche Fakultät
Universität zu Köln

Betreuung: Prof. Dr. Angela Kunothe, Samuel Leweke

Ganz herzlich bedanke ich mich bei Frau Prof. Dr. Angela Kunothe für die Vergabe des Themas und bei Samuel Leweke für die gute inhaltliche Betreuung.

Neil Vetter

Inhaltsverzeichnis

1. Einleitung	1
2. Pseudo Transient Continuation (PTC)	3
2.1. Vorüberlegungen	3
2.2. Bestimmung der optimalen Zeitschrittweite	10
2.3. Berechnung der Pseudo-Zeitschrittweite	15
3. PTC Algorithmus und seine Anwendung	18
3.1. Vorstellung des Algorithmus	18
3.2. Implementierung in Python	19
3.3. Mathematische Modellierung chemischer Systeme	22
3.4. Beispiele und Ergebnisse	24
3.4.1. Beispiel 1 – Einführendes Beispiel	24
3.4.2. Beispiel 2 – Steric Mass-Action Modell	28
3.4.3. Beispiel 3 – Komplexes pH-Puffer Modell	34
4. Robustheit des PTC Algorithmus	41
4.1. Robustheit	41
4.2. Vergleich mit einem Newton Verfahren	41
5. Fazit und Ausblick	47
Literatur	48
A. Notationsverzeichnis	49
B. Quellcode	52
B.1. Beispiel 1	52
B.2. Beispiel 2	55
B.3. Beispiel 3	59
B.4. Vergleich der Robustheit	64

Abbildungsverzeichnis

1.	Konzentrationen der Reaktanten in Beispiel 1	27
2.	Verlauf der Residuen in Beispiel 1	27
3.	Norm des Residuums in Beispiel 2	33
4.	Verlauf der Konzentrationen aus Beispiel 3	39
5.	Vergleich der beiden Verfahren	45

Tabellenverzeichnis

1.	Ergebnisse aus Beispiel 1 unter Verwendung von OPT	25
2.	Ergebnisse aus Beispiel 1 unter Verwendung von SER	26
3.	Ergebnisse aus Beispiel 2 unter Verwendung von OPT	32
4.	Ergebnisse aus Beispiel 2 unter Verwendung von SER	32
5.	Entwicklung der Residuen und Zeitschrittweiten aus Beispiel 3	39
6.	Entwicklung der Konzentrationen aus Beispiel 3	40

1. Einleitung

Diese Arbeit beschäftigt sich mit einem Verfahren zur Lösung von nichtlinearen Problemen

$$F(x) = 0, \quad F : \mathbb{R}^n \rightarrow \mathbb{R}^n. \quad (1.0.1)$$

Ein beliebtes numerisches Verfahren für die Lösung von Problemen der Art (1.0.1) ist das Newton-Verfahren. Das Verfahren ist lokal konvergent, hat also den Nachteil, dass Konvergenz zu einer Nullstelle $\tilde{x}$ nur garantiert ist, wenn der Startpunkt x_0 in einer ausreichend kleinen Umgebung um $\tilde{x}$ liegt. Der Vorteil des Newton-Verfahrens ist, dass es in dieser Umgebung quadratisch konvergiert. Es existieren verschiedene Newton-basierende Verfahren zum Lösen von nichtlinearen Problemen. Eine Einführung in Newton-Verfahren zur Lösung nichtlinearer Probleme und eine Übersicht über die verschiedenen Klassen solcher Verfahren sind in [Deu11] zu finden.

Pseudo Transient Continuation (PTC) ist ein numerisches Verfahren, das die Lösung der besprochenen Klasse von Problemen als Gleichgewichtszustand des zugehörigen Systems gewöhnlicher Differentialgleichungen (engl. „ordinary differential equations“, bzw. ODE)

$$\dot{x} = \frac{dx}{d\tau} = F(x), \quad \text{mit } x(0) = x_0, \quad (1.0.2)$$

interpretiert. Hierzu wird die Pseudo-Zeit τ eingeführt. Es wird eine Gleichgewichtslösung des Systems gesucht, das heißt (1.0.2) wird bis zu seinem Gleichgewichtspunkt integriert, an welchem die Zustandsgrößen sich nicht mehr abhängig von der Zeit verändern. Im Gegensatz zu Standard ODE-Lösern wie z.B. Newton-, Euler- oder Runge-Kutta-Verfahren, ist das Ziel von PTC nicht die Trajektorie überall adäquat aufzulösen, sondern den Gleichgewichtspunkt möglichst schnell zu erreichen. Dies geschieht mit Hilfe der Pseudo-Zeit. Sie ermöglicht große Zeitschrittweiten, die sich im Verlaufe des Verfahrens immer weiter erhöhen.

Die Arbeit besteht im Wesentlichen aus drei Teilen. Der erste Teil beschäftigt sich zunächst mit der Berechnung einer optimalen Pseudo-Zeitschrittweite τ_{opt} in jedem Iterationsschritt. Dabei wird speziell die Existenz von potentiell unbekannten dynamischen Invarianten berücksichtigt. Diese entstehen, wenn zwischen verschiedenen Komponenten des Systems Erhaltungsgrößen existieren. Sie führen zu Null-Eigenwerten in der zum Problem gehörenden Jacobi-Matrix, die somit nicht invertierbar ist. Mehr Einblicke dazu befinden sich in den Vorüberlegungen, in denen nützliche Eigenschaften von Algorithmen zur Lösung von Problemen der Art (1.0.1) erörtert werden. Neben der optimalen Zeitschrittweite werden zudem verschiedene heuristische Strategien zur Bestimmung der Pseudo-Zeitschrittweite τ betrachtet.

Der nächste Teil behandelt den PTC Algorithmus. Der Algorithmus folgt direkt aus dem mathematischen Verfahren. Zunächst wird der Algorithmus konzeptionell anhand von Pseudocode eingeführt und erklärt, und sodann unter Verwendung der Programmiersprache Python (Version 3.7) implementiert. Eine Einführung in die Programmierung mit Python für MINT-Studierende findet sich beispielsweise in [Sch19]. Für einen tieferen Einblick in das wissenschaftliche Rechnen mit Python bietet sich [Joh19] an.

Die im Algorithmus verwendeten Methoden werden im Detail erörtert. Ein möglicher Anwendungsbereich für den Algorithmus sind chemische Systeme, deren Gleichgewichtszustand zu ermitteln ist. In solchen Systemen existieren häufig Beziehungen zwischen den reagierenden Molekülen, die in einem mathematischen System als Erhaltungsgrößen auftauchen.

Um den Algorithmus für die Lösung solcher Probleme anzuwenden, wird darauf eingegangen, wie ein chemisches System mathematisch modelliert wird. Anschließend wird der Algorithmus für verschiedene Beispiele angewendet. Zu jedem Beispiel werden die chemischen Hintergründe erörtert und gezeigt wie das zugehörige mathematisches Modell konstruiert wird, welches der Algorithmus anschließend löst. Im Anschluss wird analysiert, welche Auswirkung die Wahl der Zeitschrittweite auf den Verlauf des Verfahrens hat. Zu diesem Zweck werden verschiedene Strategien zur Berechnung der Zeitschrittweite getestet und jeweils im Hinblick auf die Unterschiede in ihrem Konvergenz-Verhalten verglichen.

Der letzte Teil der Arbeit beschäftigt sich mit der Robustheit des PTC Algorithmus in Abhängigkeit von der Entfernung des Startpunktes x_0 zum Gleichgewichtspunkt x_{ref} . Hierfür wird das Verfahren für eine große Anzahl an gleich-verteilten Punkten mit selbem Abstand zu x_{ref} durchgeführt. Dieser Abstand wird schrittweise erhöht und so der Prozess für immer neue Punkte durchgeführt. Überprüft wird, für wie viele der Punkte in jedem Schritt das Verfahren konvergiert. Zum Vergleich dient das Newton-Verfahren NLEQ_RES. Es wird gezeigt, dass die Ergebnisse des PTC Algorithmus, im Vergleich zum Newton-Verfahren, weniger abhängig von der Wahl des Startpunktes sind.

2. Pseudo Transient Continuation (PTC)

2.1. Vorüberlegungen

In diesem Kapitel betrachten wir die Lösung nichtlinearer Systeme

$$F(x) = 0,$$

die als Gleichgewichtsproblem des Systems gewöhnlicher Differentialgleichungen

$$\dot{x} = F(x), \quad \text{mit } x(0) = x_0 \quad (2.1.1)$$

interpretiert werden. Die Beobachtungen und Beweise in diesem Kapitel orientieren sich an Kapitel 2 und Kapitel 6 aus [Deu11]. Es folgen nützliche Begriffe für die Konstruktion von Algorithmen zur Lösung von Problemen der Art (2.1.1). Als erstes affine Ähnlichkeit, die eine Invarianz-Klasse beschreibt. Essentiell ist hier, dass Pseudo Transient Continuation als Algorithmus dieser Klasse konstruiert wird. Mit der Invarianz-Klasse kann dann die lineare Kontraktion von Systemen behandelt werden. In diesem Abschnitt wird eine Konstante eingeführt, die später als einseitige Lipschitz-Bedingung wieder auftaucht, wenn wir die Berechnung der Zeitschrittweiten genau betrachten. Im Anschluss wird der Begriff dynamische Invariante eingeführt. Dabei handelt es sich um eine Klasse von Invarianten, die in den behandelten Problemen häufig auftreten und zu singulären Jacobi-Matrizen $F'(x)$ führen. Ihre Existenz muss später bei der Berechnung der Zeitschrittweiten beachtet werden.

Affine Ähnlichkeit

Affine Ähnlichkeit ist eine von vier Invarianz-Klassen die in Kapitel 2 aus [Deu11] beschrieben werden.

Definition 2.1. Ein Problem der Art (2.1.1) heißt affin ähnlich, wenn bei einer Transformation des Problems, sowohl die Definitionsmenge als auch das Bild von F auf dieselbe Weise transformiert werden.

Um die Eigenschaft zu veranschaulichen, wird für die Klasse von Problemen (2.1.1) eine affine Transformation mit einer beliebigen invertierbaren Matrix B durchgeführt, so dass

$$B\dot{x} = BF(x) = 0.$$

Mit $y = Bx$ ergibt sich das transformierte Problem

$$G(y) := BF(B^{-1}y) = \dot{y}, \quad \text{mit } y_0 = Bx_0. \quad (2.1.2)$$

Hier wird sowohl die Definitionsmenge als auch das Bild von F auf dieselbe Weise transformiert. Für die Jacobi-Matrix des transformierten Problems (2.1.2) ergibt sich die namensgebende Ähnlichkeitstransformation

$$G'(y) = BF'(B^{-1}y) = BF'(x)B^{-1}. \quad (2.1.3)$$

Bemerkung 2.1. Offensichtlich hat die obige Ähnlichkeitstransformation keinen Einfluss auf die Eigenwerte der Jacobi-Matrix. Eine obere Schranke der Eigenwerte wird später bei der Bestimmung einer einseitigen Lipschitz-Bedingung eine Rolle spielen. Diese wird benötigt, da die Lipschitz-Konstante von F die Betrachtung von steifen Anfangswertproblemen nicht zulässt.

Sei J die Jordan-Normalform von $F'(x)$, bestehend aus Jordanblöcken für alle verschiedenen Eigenwerte, dann ist die Jacobi-Matrix an jedem Punkt x darstellbar als

$$F'(x) = T(x)J(x)T^{-1}(x) = TJT^{-1}.$$

Mit (2.1.3) bedeutet das für das transformierte Problem

$$G'(y) = BF'(x)B^{-1} = (BT)J(BT)^{-1}.$$

Bemerkung 2.2. Das bedeutet, dass Aussagen die mit Hilfe der kanonischen Norm $|u| := \|T^{-1}u\|$ getroffen werden, automatisch die Eigenschaft der affinen Ähnlichkeit realisieren. Das bedeutet, dass Aussagen die in dieser Norm getroffen werden invariant gegenüber Transformation mit einem B , wie zuvor definiert, sind.

Bemerkung 2.3. Sei B wieder eine beliebige invertierbare Matrix mit der (2.1.1) transformiert wird, so dass

$$G(y) := BF(B^{-1}y) = 0, \text{ mit } y = Bx \text{ und } y_0 = Bx_0.$$

Wird das Newton-Verfahren zur Lösung des transformierten Problems $G(y) = 0$ verwendet, ergibt sich

$$G'(y^k)\Delta y^k = -G(y^k), \quad y^{k+1} = y^k + \Delta y^k, \quad \text{für } k = 0, 1, \dots$$

Mit $G'(y) = BF'(x)B^{-1}$ und $y_0 = Bx_0$ folgt direkt

$$x^k = B^{-1}y^k \quad \text{und} \quad \Delta x^k = B^{-1}\Delta y^k.$$

Das gleiche gilt ebenso für Fixpunkt-Iterationen

$$\Delta y^k = G(y^k), \quad y^{k+1} = y^k + \Delta y^k.$$

Newton-Iterationen der Form

$$F'(x^k)\Delta x^k = -F(x^k), \quad x^{k+1} = x^k + \gamma_k \Delta x^k, \quad \text{für } k = 0, 1, \dots, \quad (2.1.4)$$

sowie Fixpunkt-Iterationen der Form

$$\Delta x^k = F(x^k), \quad x^{k+1} = x^k + \gamma_k \Delta x^k, \quad \text{für } k = 0, 1, \dots, \quad (2.1.5)$$

sind affin ähnlich, mit einer lokalen Schrittweite γ_k , die in jeder Iteration neu angepasst wird. Die obigen Iterationen haben den Nachteil, dass sie nur für die Lösung nicht-steifer ODE geeignet sind. Zur Lösung von Problemen denen steife ODE zugrunde liegen eignet sich das sogenannte Pseudo Transient Continuation Verfahren

$$(I - \tau F'(x^k))\Delta x^k = F(x^k), \quad x^{k+1} = x^k + \tau \Delta x^k, \quad \text{für } k = 0, 1, \dots \quad (2.1.6)$$

mit einem Pseudo-Zeitschritt τ , der in jeder Iteration angepasst wird. $F'(x)$ ist die Jacobi-Matrix. Das Verfahren ergibt sich als lineare Kombination der beiden Iterationen (2.1.4) und (2.1.5). Als solche besitzt das Verfahren ebenfalls die Eigenschaft der affinen Ähnlichkeit. Es ist ein linear-implizites Euler-Verfahren für das System (2.1.1). Für sehr große Zeitschrittweiten ($\tau \rightarrow \infty$) geht das obige Verfahren in das Newton-Verfahren über.

Lineare Kontraktion

Die in diesem Abschnitt getroffenen Aussagen werden später dazu verwendet eine einseitige Lipschitz-Bedingung zu formulieren, die sich über die Eigenwerte der Jacobi-Matrix definieren lässt. Außerdem zeigen wir eine nützliche Eigenschaft der Bedingung, mit der sich Reduktion des Residuums in den Iterationsschritten garantieren lässt.

Es sei eine lineare gewöhnliche Differentialgleichung der Form

$$\dot{x} = Ax, \quad x(0) = x_0, \quad \text{mit } A \in \mathbb{R}^{n \times n} \quad (2.1.7)$$

gegeben. Probleme dieser Art besitzen, unter Verwendung des Matrixexponentials, die Lösung

$$x(t) = \exp(At)x_0.$$

Eine Koordinatentransformation $y = Bx$ mit einer beliebigen invertierbaren Matrix B stellt eine Ähnlichkeitstransformation

$$A \rightarrow BAB^{-1}$$

dar. Sei J die reelle Jordan-Normalform von A , bestehend aus Jordanblöcken für alle verschiedenen reellen Eigenwerte $\lambda(A)$. Dann folgt äquivalent die Transformation

$$A = TJT^{-1} \rightarrow (BT)J(BT)^{-1}. \quad (2.1.8)$$

Sei die euklidische Norm $\|\cdot\|$ mit Skalarprodukt $(u, v) = u^T v$ und die kanonische Norm $|\cdot|$ mit Skalarprodukt $\langle u, v \rangle = (T^{-1}u, T^{-1}v)$ definiert. Wie in Bemerkung 2.2 gilt, dass alle Aussagen die unter der so skalierten kanonischen Norm getroffen werden, automatisch die Eigenschaft der affinen Ähnlichkeit realisieren.

Sei $\mu = \mu(A)$ eine Konstante, die so definiert ist, dass

$$\langle u, Au \rangle \leq \mu(A)|u|^2, \quad \forall u \in \mathbb{R}^n$$

gilt. Das ist mit (2.1.8) offensichtlich äquivalent zu

$$(\tilde{u}, J\tilde{u}) \leq \mu\|\tilde{u}\|^2, \quad \forall \tilde{u} \in \mathbb{R}^n, \quad (2.1.9)$$

mit $\tilde{u} = T^{-1}u$.

Satz 2.1. Wird μ optimal gewählt, ist die Ungleichung

$$\mu(A) = \max_{u \neq 0} \frac{\langle u, Au \rangle}{|u|^2} \geq \max_i \lambda_i(A) + \epsilon \quad \text{mit } \epsilon \geq 0 \quad (2.1.10)$$

erfüllt. Wenn der dominierende Eigenwert von A einfach ist, gilt Gleichheit und $\epsilon = 0$.

Bemerkung 2.4. Laut [Deu11] kann eine äquivalente Abschätzung auch für komplexe Eigenwerte gemacht werden. In diesem Fall steht auf der rechten Seite der Eigenwert mit dominierendem Realteil

$$\max_i \operatorname{Re}(\lambda_i(A)).$$

Er wird mit Spektralabszisse der Matrix bezeichnet.

Beweis des Satzes. Mit (2.1.9) gilt

$$\mu(A) = \max_{u \neq 0} \frac{\langle u, Au \rangle}{|u|^2} = \max_{\tilde{u} \neq 0} \frac{(\tilde{u}, J\tilde{u})}{\|\tilde{u}\|^2}.$$

J befindet sich in Jordan Normalform. Also ist J Blockdiagonal, mit einem Block für jeden Eigenwert λ_i .

Das Maximum über alle Vektoren $\tilde{u} \neq 0$ ist sicher größer-gleich dem Wert eines beliebigen dieser Vektoren. Sei o.B.d.A. der größte Eigenwert $\lambda_{\max}$ der Matrix A der i -te Eigenwert λ_i .

Wir wählen einen Einheitsvektor, der seinen einzigen Eintrag in der i -ten Zeile hat.

1.Fall: $\lambda_{\max}$ ist ein einfacher Eigenwert.

In diesem Fall hat der zu $\lambda_{\max}$ gehörende Block in J nur einen Eintrag, welcher gleichzeitig der einzige Eintrag in der i -ten Spalte ist. Dann ist

$$\mu(A) = \max_{\tilde{u} \neq 0} \frac{(\tilde{u}, J\tilde{u})}{\|\tilde{u}\|^2} \geq \max_i \lambda_i(A).$$

Außerdem ist

$$\begin{aligned} \max_{\tilde{u} \neq 0} \frac{(\tilde{u}, J\tilde{u})}{\|\tilde{u}\|^2} &= \frac{\lambda_1 \tilde{u}_1^2 + \dots + \lambda_n \tilde{u}_n^2}{\tilde{u}_1^2 + \dots + \tilde{u}_n^2} \\ &\leq \frac{\lambda_{\max}(\tilde{u}_1^2 + \dots + \tilde{u}_n^2)}{\tilde{u}_1^2 + \dots + \tilde{u}_n^2} \\ &= \lambda_{\max} = \max_i \lambda_i(A). \end{aligned}$$

Bei einem einfachen größtem Eigenwert gilt also

$$\mu(A) = \max_{\tilde{u} \neq 0} \frac{(\tilde{u}, J\tilde{u})}{\|\tilde{u}\|^2} = \max_i \lambda_i(A).$$

2.Fall: $\lambda_{\max}$ ist kein einfacher Eigenwert.

Hier ist der zu $\lambda_{\max}$ gehörende Block größer-gleich (2×2) , mit einer oberen Nebendiagonale aus Einsen. Es genügt bei größeren Blöcken den linken oberen (2×2) Unterblock zu betrachten.

Sei $\tilde{u} \in \mathbb{R}^n$ mit $\|\tilde{u}\|^2 = 1$, sei $\kappa \in (0, 1)$ und

$$\tilde{u} := \begin{pmatrix} \vdots \\ 0 \\ \kappa \\ \sqrt{1 - \kappa^2} \\ 0 \\ \vdots \end{pmatrix}.$$

Dann ergibt sich

$$\begin{aligned}
(\tilde{u}, J\tilde{u}) &= \begin{pmatrix} \kappa \\ \sqrt{1-\kappa^2} \end{pmatrix}^T \begin{pmatrix} \lambda & 1 \\ 0 & \lambda \end{pmatrix} \begin{pmatrix} \kappa \\ \sqrt{1-\kappa^2} \end{pmatrix} \\
&= \begin{pmatrix} \kappa \\ \sqrt{1-\kappa^2} \end{pmatrix}^T \begin{pmatrix} \lambda\kappa + \sqrt{1-\kappa^2} \\ \lambda\sqrt{1-\kappa^2} \end{pmatrix} \\
&= \lambda\kappa^2 + \kappa\sqrt{1-\kappa^2} + \lambda(1-\kappa^2) \\
&= \lambda + \kappa\sqrt{1-\kappa^2}.
\end{aligned}$$

Und somit insgesamt

$$\mu(A) = \max_{\tilde{u} \neq 0} \frac{(\tilde{u}, J\tilde{u})}{\|\tilde{u}\|^2} \geq \max_i \lambda_i(A) + \kappa\sqrt{1-\kappa^2} = \max_i \lambda_i(A) + \epsilon$$

mit $\epsilon = \kappa\sqrt{1-\kappa^2}$. □

Mit Satz 2.1 und der affinen Transformation (2.1.8) gilt

$$\mu(BAB^{-1}) = \mu(A), \quad (2.1.11)$$

was noch einmal die affine Ähnlichkeit von μ bestätigt. Damit lässt sich die Abschätzung

$$|x(t)| \leq \exp(\mu t)|x_0| \quad \text{für } t \geq 0$$

erhalten. Wenn zusätzlich $\mu < 0$ gilt, ist

$$|x(t)| < |x_0| \quad \text{für } t > 0.$$

Daraus folgt, dass das ursprüngliche System (2.1.7) kontrahiert.

Das kanonische Skalarprodukt garantiert affine Ähnlichkeit, ist aber schwierig zu berechnen und anfällig für schlechte Konditionierung der Matrix T . Aus diesen Gründen wird in der Praxis häufig das euklidische Skalarprodukt verwendet. Damit geht die Eigenschaft der affinen Ähnlichkeit verloren, welche durch Skalierung der Norm wiederhergestellt werden kann. Sei mit Hinblick auf Satz 2.1 eine neue Konstante definiert, nur diesmal unter Verwendung der euklidischen Norm

$$\nu(A) = \max_{u \neq 0} \frac{(u, Au)}{\|u\|^2}. \quad (2.1.12)$$

In diesem Fall kann ν äquivalent mit

$$\nu(A) = \lambda_{\max} \left(\frac{1}{2}(A + A^T) \right)$$

ausgedrückt werden. Dabei ist $\lambda_{\max}$ der maximale reelle Eigenwert des symmetrischen Teils der Matrix A . Durch einen Vergleich mit (2.1.9) folgt

$$\mu(A) = \nu(J) = \lambda_{\max} \left(\frac{1}{2}(J + J^T) \right),$$

was direkt zur Aussage von Satz 2.1 führt.

Die Werte für $\nu(A)$ und $\mu(A)$ unterscheiden sich im Allgemeinen. Im Gegensatz zu (2.1.11) gilt für eine beliebige Transformationsmatrix B im Allgemeinen

$$\nu(BAB^{-1}) \neq \nu(A).$$

Somit ist ν nicht affin ähnlich.

Zusammengefasst zeigt sich, dass Kontraktion in der Differentialgleichung zu Kontraktion in der kanonischen Norm $|\cdot|$ führt, jedoch nicht unbedingt zu Kontraktion in der euklidischen Norm $\|\cdot\|$ führen muss.

Mit $\nu(A) < 0$ lässt sich die Abschätzung

$$\|x(t)\| \leq \exp(\nu t) \|x_0\| < \|x_0\| \quad \text{für } t > 0 \quad (2.1.13)$$

machen.

Bemerkung 2.5. Im Allgemeinen kann von einer Beziehung

$$|u| \leq |v|$$

bezüglich der kanonischen Norm nur auf eine Beziehung

$$\|u\| \leq \text{cond}(T) \|v\|$$

in der euklidischen Norm geschlossen werden. Die Konditionszahl $\text{cond}(T) = \|T^{-1}\| \cdot \|T\| \geq 1$ ergibt sich als geometrischer Störfaktor, der seinen Ursprung in einer schlechten Konditionierung der Jordan-Zerlegung hat.

Dynamische Invarianten

In dynamischen Systemen treten diese Klasse von Invarianten häufig auf und führen zu einer singulären Jacobi-Matrix $F'(x)$ für alle x . Um dies besser zu verdeutlichen, betrachten wir das Beispiel der Massenerhaltung.

Beispiel 2.1. Massenerhaltung tritt in chemischen Systemen häufig auf und besagt, dass die gesamte Masse der im System enthaltenen Stoffe konstant bleibt. Es gilt also

$$e^T x(t) = e^T x_0$$

mit $e^T = (1, \dots, 1)$ für alle t . Daraus folgt für die Konzentrationen der Stoffe

$$\begin{aligned} x_1(t) + x_2(t) + \dots + x_n(t) &= \text{konstant für alle } t \\ \Rightarrow \dot{x}_1 + \dot{x}_2 + \dots + \dot{x}_n &= 0. \end{aligned}$$

Das impliziert für $F : D \rightarrow \mathbb{R}^n$

$$e^T \dot{x} = e^T F(x) = 0 \quad \text{für alle } x \in D \subset \mathbb{R}^n \text{ mit } F(x) \neq 0.$$

Durch Differenzieren nach x ergibt sich für die Jacobi-Matrix

$$e^T F'(x) F(x) = 0 \quad \text{für alle } x \in D \subset \mathbb{R}^n \text{ mit } F(x) \neq 0. \quad (2.1.14)$$

Da $F(x) \neq 0$ ist, besitzt $F'(x)$ einen Null-Eigenwert mit linkem Eigenvektor e und ist nicht invertierbar. In diesem Fall kann kein standard Newton-Verfahren zur Lösung von (2.1.1) verwendet werden.

Sei die Orthogonal-Projektion $P^\perp$ definiert als

$$P^\perp := \frac{1}{n} e e^T, \quad P = I - P^\perp.$$

Dann gilt äquivalent zu (2.1.14), dass

$$P^\perp F'(x) = 0 \quad \text{für alle } x \in D. \quad (2.1.15)$$

Bemerkung 2.6. Sind, wie in diesem Fall, alle dynamischen Invarianten bekannt, kann das Newton-Verfahren an diese angepasst werden. Für das Beispiel der Massenerhaltung mit besprochener Invariante $e^T x(t)$ ergibt sich

$$F'(x^k) \Delta x^k = -F(x^k), \quad e^T \Delta x^k = 0$$

als geeignete Anpassung.

Im Allgemeinen können mehrere, unbekannte dynamische Invarianten existieren. Das bedeutet, es existiert eine unbekannte Projektion P , so dass wieder (2.1.15) mit

$$P^\perp \dot{x} = P^\perp F(x) = 0 \quad \Rightarrow \quad P^\perp F'(x) = 0 \quad (2.1.16)$$

gilt. Die beiden Iterationen (2.1.5) und (2.1.6) realisieren für ein $P^\perp F(x) = 0$ automatisch die Eigenschaft

$$P^\perp \Delta x = 0.$$

Vorausgesetzt es gilt ebenfalls $P^\perp x_0 = 0$, dann bleiben alle Iterierten innerhalb des Unterraums

$$S_P = \{u \in \mathbb{R}^n \mid P^\perp u = 0\}.$$

2.2. Bestimmung der optimalen Zeitschrittweite

Im Folgenden betrachten wir eine Iteration von Pseudo Transient Continuation (2.1.6) mit einer Zeitschrittweite τ , beginnend bei einem Startpunkt x_0 . Wir behandeln $\tau \geq 0$ als kontinuierliche Variable, um die optimale Zeitschrittweite zu bestimmen. Die Verfahrensvorschrift lautet

$$(I - \tau F'(x_0))\Delta x = F(x_0), \quad x(\tau) = x_0 + \tau \Delta x. \quad (2.2.1)$$

Voraussetzung ist die Existenz einer exakt bestimmbaren Jacobi-Matrix $F'(x_0)$ sowie die Lösbarkeit des linearen Systems (2.2.1) mit Hilfe von exakten Eliminationsmethoden. Für die Jacobi-Matrix wird $F'(x_0) = A$ geschrieben.

Die Abbildung $x(\tau)$ beschreibt durch (2.2.1) einen Weg beginnend beim Startpunkt x_0 . Das folgende Lemma beschreibt das Residuum entlang dieses Wegs und hilft später dabei eine Reduktion des genormten Residuums $\|F(x(\tau))\|$ zu beweisen.

Lemma 2.1. *Das Residuum entlang des Wegs $x(\tau)$ mit Startpunkt x_0 erfüllt*

$$F(x(\tau)) = (I - \tau A)^{-1}F(x_0) + \int_0^\tau (F'(x(\sigma)) - A)(I - \sigma A)^{-2}F(x_0)d\sigma. \quad (2.2.2)$$

Beweis. Unter Verwendung des Hauptsatzes der Differential- und Integralrechnung folgt

$$F(x(\tau)) = F(x_0) + \int_0^\tau F'(x(\sigma))\dot{x}(\sigma)d\sigma.$$

Mit $A(x(\tau) - x_0) = \int_0^\tau A\dot{x}(\sigma)d\sigma$ folgt

$$F(x(\tau)) = F(x_0) + A(x(\tau) - x_0) + \int_0^\tau (F'(x(\sigma)) - A)\dot{x}(\sigma)d\sigma.$$

Durch differenzieren der Homotopie (2.2.1) nach τ ergibt sich

$$\begin{aligned} \dot{x} &= (I - \tau A)^{-1}F(x_0) + \tau(I - \tau A)^{-2}AF(x_0) \\ &= (I - \tau A)^{-1}(I + \tau A(I - \tau A)^{-1})F(x_0) \\ &= (I - \tau A)^{-1}(I + (\tau A + I - I)(I - \tau A)^{-1})F(x_0) \\ &= (I - \tau A)^{-1}(I - I + (I - \tau A)^{-1})F(x_0) \\ &= (I - \tau A)^{-2}F(x_0). \end{aligned}$$

Mit (2.2.1) folgt außerdem $\tau(I - \tau A)^{-1}F(x_0) = x(\tau) - x_0$. Daraus lässt sich auf

$$(I - \tau A)\dot{x} = F(x_0) + \tau A(I - \tau A)^{-1}F(x_0) = F(x_0) + A(x(\tau) - x_0)$$

schließen und somit mit $\dot{x} = (I - \tau A)^{-2}F(x_0)$ auf

$$F(x_0) + A(x(\tau) - x_0) = (I - \tau A)^{-1}F(x_0).$$

Einsetzen für $\dot{x}$ und $F(x_0) + A(x(\tau) - x_0)$ beendet den Beweis des Lemmas. □

Einseitige Lipschitz-Bedingung erster Ordnung

Bemerkung 2.7. Laut [Deu11] eignet sich eine einseitige Lipschitz-Bedingung hier besser, da eine normale Lipschitz-Bedingung zu Problemen mit steifen Gleichungssystemen führen kann.

Wie in Abschnitt 2.1 gezeigt, sollte das dem Gleichgewichtsproblem zugrunde liegende dynamische System (2.1.1) als eine Differential Gleichung interpretiert werden. Wie schon in Bemerkung 2.1 festgehalten, kann keine Lipschitz-Bedingung erster Ordnung von F verwendet werden, da sich diese nur für nicht-steife Probleme eignet. Aus diesem Grund wird eine einseitige Lipschitz-Bedingung erster Ordnung konstruiert.

Wie bereits in (2.1.12) definiert sei $\nu = \nu(A)$ eine Konstante, so dass

$$(u, Au) \leq \nu \|u\|^2 \text{ mit } u \in S_P. \quad (2.2.3)$$

Wenn im System $F(x)$ dynamische Invarianten existieren, besitzt die Jacobi-Matrix A Null-Eigenwerte und ist nicht invertierbar. Das impliziert laut Satz 2.1 $\nu \geq 0$ und nicht $\nu < 0$, wie in (2.1.13) vorausgesetzt. Um diesen Fall zu vermeiden, beschränken wir uns aufgrund der Ergebnisse aus Abschnitt 2.1 auf Korrekturen Δx im Unterraum

$$S_P = \{u \in \mathbb{R}^n | P^\perp u = 0\}.$$

Dann ist (2.2.3) äquivalent zu

$$(Pu, (PAP)Pu) \leq \nu \|Pu\|^2$$

mit geeigneter Projektion P . In dieser Definition gilt $\nu < 0$ auch wenn dynamische Invarianten auftreten, solange sich die Iterierten auf den Unterraum S_P beschränken.

Da $\Delta x(\tau)$ ein Element in S_P ist, wird es in (2.2.3) eingesetzt und es folgt

$$\tilde{\nu}(\tau) = \frac{(\Delta x, A\Delta x)}{\|\Delta x\|^2} \leq \nu. \quad (2.2.4)$$

Wird das Problem (2.2.1) von links mit $(\Delta x)^T$ multipliziert, ergibt sich mit (2.2.4)

$$\begin{aligned} \|\Delta x\|^2 &= \tau(\Delta x, A\Delta x) + (\Delta x, F(x_0)) \\ &= \tau\tilde{\nu}(\tau)\|\Delta x\|^2 + (\Delta x, F(x_0)) \\ &\leq \tau\tilde{\nu}(\tau)\|\Delta x\|^2 + \|\Delta x\|\|F(x_0)\| \\ &\leq \tau\nu\|\Delta x\|^2 + \|\Delta x\|\|F(x_0)\|. \end{aligned}$$

Die letzten beiden Ungleichungen können umgestellt werden, um auf das Verhältnis

$$\|\Delta x\| \leq \frac{\|F(x_0)\|}{1 - \tilde{\nu}\tau} \leq \frac{\|F(x_0)\|}{1 - \nu\tau} \quad (2.2.5)$$

zu schließen. Da $\Delta x(\tau) = F(x_0) + \mathcal{O}(\tau)$ ist, ergibt Einsetzen in (2.2.4) mit $\tau = 0$

$$\tilde{\nu}(0) = \frac{(F(x_0), AF(x_0))}{\|F(x_0)\|^2} \leq \nu. \quad (2.2.6)$$

Der Wert für $\tilde{\nu}(0)$ kann schon vor dem Lösen des linearen Systems (2.2.1) betrachtet werden. Er spielt eine wichtige Rolle für die Betrachtung der Reduktion des Residuums, wie im folgenden Lemma erkenntlich wird.

Lemma 2.2. Sei $\tilde{\nu}(0) < 0$ wie in (2.2.6) definiert. Dann existiert ein $\tau^* > 0$, so dass

$$\|F(x(\tau))\| < \|F(x_0)\| \text{ und } \tilde{\nu}(\tau) < 0 \text{ für alle } \tau \in [0, \tau^*).$$

Beweis. Durch Differenzieren des Residuums nach τ ergibt sich

$$\begin{aligned} \frac{d}{d\tau} \|F(x(\tau))\|^2|_{\tau=0} &= 2(F'(\cdot)^T F(\cdot), \dot{x}(\tau))|_{\tau=0} \\ &= 2(F(x_0), AF(x_0)) \\ &= 2\tilde{\nu}(0)\|F(x_0)\|^2 < 0. \end{aligned}$$

Da sowohl $F(x(\tau))$ als auch die Norm $\|\cdot\|$ stetig differenzierbar sind, existiert ein nicht-leeres Intervall bezüglich τ , innerhalb dessen das Residuum abnimmt. Der Beweis für $\tilde{\nu}(\tau)$ verläuft analog. $\square$

Das bedeutet, dass der Wert für $\tilde{\nu}(0)$ schon vor Beginn des Verfahrens am Startpunkt x_0 beobachtet werden kann und falls dieser bereits die Bedingung $\tilde{\nu}(0) < 0$ nicht erfüllt, keine Reduktion des Residuums bei Durchführung von Pseudo Transient Continuation erwartet wird.

Lipschitz-Bedingung zweiter Ordnung

Sei $F : D \rightarrow \mathbb{R}$ eine Funktion. Dann definieren wir eine affin ähnliche Lipschitz-Bedingung

$$|(F'(x) - F'(y))u| \leq L \|x - y\| |u|, \text{ mit } x, y \in D \text{ und } u \in S_P$$

und ersetzen die kanonische Norm $|\cdot|$ mit der euklidischen Norm $\|\cdot\|$. Damit erhalten wir

$$\|(F'(x) - F'(y))u\| \leq L \|x - y\| \|u\|, \text{ mit } x, y \in D \text{ und } u \in S_P \quad (2.2.7)$$

als eine geeignete Lipschitz-Bedingung zweiter Ordnung.

Mit Lemma 2.1 und den beiden Lipschitz-Bedingungen sind alle Vorbereitungen getroffen, um die optimale Zeitschrittweite für einen Iterationsschritt zu bestimmen.

Satz für die (theoretisch) optimale Zeitschrittweite

Satz 2.2. Sei $F_0 := \|F(x_0)\|$ und immer noch $A = F'(x_0)$ wie zuvor. Dynamische Invarianten werden durch die Eigenschaft $F(x) \in S_P$ behandelt. Zusätzlich wird die einseitige Lipschitz-Bedingung erster Ordnung

$$(u, Au) \leq \nu \|u\|^2 \text{ für } u \in S_P \text{ mit } \nu < 0,$$

sowie die Lipschitz-Bedingung zweiter Ordnung

$$\|(F'(x) - F'(x_0))u\| \leq L \|x - x_0\| \|u\|$$

angenommen. Unter diesen Bedingungen gilt die Ungleichung

$$\|F(x(\tau))\| \leq \left(1 + \frac{1}{2} \frac{F_0 L \tau^2}{1 - \nu \tau}\right) \frac{F_0}{1 - \nu \tau}. \quad (2.2.8)$$

Daraus folgt, dass für alle $\tau \geq 0$, welche die Bedingung

$$\nu + \left(\frac{1}{2}F_0 L - \nu^2\right)\tau \leq 0 \quad (2.2.9)$$

erfüllen, Monotonie des Residuums

$$\|F(x(\tau))\| \leq \|F(x_0)\|$$

garantiert ist. Wenn zusätzlich die Bedingung

$$F_0 L > \nu^2 \quad (2.2.10)$$

erfüllt ist, ist die (theoretisch) optimale Pseudo-Zeitschrittweite mit

$$\tau_{\text{opt}} = \frac{|\nu|}{F_0 L - \nu^2} \quad (2.2.11)$$

gegeben. Das führt zur Reduktion des Residuums in der Form

$$\|F(x(\tau))\| \leq \left(1 - \frac{1}{2} \frac{\nu^2}{F_0 L}\right) \|F(x_0)\| < \|F(x_0)\|.$$

Beweis. Zunächst wird die Gleichung (2.2.2) aus Lemma 2.1 betrachtet. Offensichtlich sind der vordere und hintere Term der rechten Seite unabhängig voneinander und können getrennt gezeigt werden. Laut Lemma 2.2 ist die Bedingung $\nu < 0$ Voraussetzung dafür, dass Reduktion des Residuums vorliegen kann. Für den vorderen Term bedeutet das, dass für den folgenden Beweis $\nu := -|\nu|$ definiert sein muss.

Bemerkung 2.8. Für den hinteren Term folgern wir aus der Verfahrensvorschrift (2.2.1) und unter Verwendung des Verhältnis (2.2.5)

$$x(\tau) = x_0 + \tau \Delta x \Rightarrow \|x(\tau) - x_0\| = \tau \|\Delta x\| \leq \frac{\tau \|F(x_0)\|}{1 - \tau \nu}.$$

Zusätzlich gilt offensichtlich

$$\|(I - \sigma A)^{-2} F(x_0)\| \leq \|(I - \sigma A)^{-2}\| \|F(x_0)\|.$$

Daraus folgt

$$\|(I - \tau A)^{-1}\| \leq \frac{1}{1 - \tau \nu}$$

und somit

$$\|(I - \tau A)^{-2}\| \leq \frac{1}{(1 - \tau \nu)^2}.$$

Mit dem Ergebnis dieser Bemerkung und unter Verwendung der Lipschitz-Bedingungen (2.2.7) und (2.2.6) wird das Integral auf der rechten Seite abgeschätzt mit

$$\begin{aligned} & \left\| \int_0^\tau (F'(x(\sigma)) - A)(I - \sigma A)^{-2} F(x_0) d\sigma \right\| \\ & \leq L \int_0^\tau \|x(\sigma) - x_0\| \|(I - \sigma A)^{-2} F(x_0)\| d\sigma \\ & \leq L \int_0^\tau \frac{\sigma \|F(x_0)\|^2}{(1 - \sigma \nu)^3} d\sigma \\ & = \frac{1}{2} L \|F(x_0)\|^2 \tau^2 (1 - \nu \tau)^{-2}. \end{aligned} \quad (2.2.12)$$

Dies bestätigt Aussage (2.2.8) des Satzes für

$$\|F(x(\tau))\| \leq \alpha(\tau) \|F(x_0)\|$$

mit

$$\alpha(\tau) = \frac{1 - \nu\tau + \frac{1}{2}F_0 L \tau^2}{(1 - \nu\tau)^2}. \quad (2.2.13)$$

Wird α zusätzlich mit $\alpha(\tau) \leq 1$ eingeschränkt, gilt ebenso die äquivalente Aussage (2.2.9).

Durch Differenzieren von $\alpha(\tau)$ nach τ ergibt sich

$$\dot{\alpha}(\tau) = \left(\nu + \frac{F_0 L \tau}{1 - \nu\tau} \right) / (1 - \nu\tau)^2.$$

Eine optimale Reduktion des Residuums erfolgt am Minimum von α , also $\dot{\alpha}(\tau) = 0$. Die optimale Zeitschrittweite τ_{opt} ist dann durch (2.2.11) gegeben, vorausgesetzt Bedingung (2.2.10) gilt. Einsetzen des so bestimmten optimalen Zeitschritts in die Definition (2.2.13) beendet den Beweis. $\square$

Bemerkung 2.9. Bedingung (2.2.9) erlaubt folgende Aussagen über die Zeitschrittweite τ . Falls

$$\nu + \frac{1}{2}\sqrt{2F_0 L} \leq 0,$$

ist τ unbeschränkt. Gilt jedoch

$$\nu + \frac{1}{2}\sqrt{2F_0 L} > 0, \quad (2.2.14)$$

ist die Zeitschrittweite beschränkt mit

$$\tau \leq \frac{|\nu|}{\frac{1}{2}F_0 L - |\nu|^2}.$$

Bedingung (2.2.10) schränkt weniger ein als Bedingung (2.2.14), so dass sowohl beschränkte als auch unbeschränkte Zeitschrittweiten möglich sind.

2.3. Berechnung der Pseudo-Zeitschrittweite

Sei wie zuvor $F_0 = \|F(x_0)\|$. Und seien ν sowie L Lipschitz-Konstanten, wie im Kapitel 2.2 konstruiert. Unter Verwendung dieser Werte soll eine adaptive Strategie für die Wahl der Zeitschrittweite gefunden werden. Es wird eine Approximation der optimalen Zeitschrittweite τ_{opt} gesucht, die mit möglichst geringem rechnerischem Aufwand berechnet werden kann. Zu diesem Zweck werden rechnerische Näherungen für die beiden Lipschitz-Bedingungen ν und L benötigt. Zunächst sei noch einmal die Formel für die theoretisch optimale Zeitschrittweite

$$\tau_{\text{opt}} = \frac{|\nu|}{F_0 L - \nu^2} \quad (2.3.1)$$

gegeben. Die Formel kann in ihre implizite Form

$$\tau_{\text{opt}} = \frac{|\nu|(1 - \nu\tau_{\text{opt}})}{F_0 L} \quad (2.3.2)$$

gebracht werden. Unter Verwendung von Verhältnis (2.2.5) folgt daraus

$$\|\Delta x(\tau_{\text{opt}})\| L \tau_{\text{opt}} \leq \frac{F_0 L \tau_{\text{opt}}}{1 - \nu\tau_{\text{opt}}} = |\nu|.$$

Durch Umstellen ergibt sich mit

$$\bar{\tau}_{\text{opt}} := \frac{|\nu|}{L \|\Delta x(\tau_{\text{opt}})\|} \geq \tau_{\text{opt}}$$

eine obere Schranke für τ_{opt} .

Es müssen also einfach zu bestimmende Nährwerte $[\nu] \leq \nu < 0$ und $[L] \leq L$ für die beiden Lipschitz-Bedingungen gefunden werden. Mit diesen beiden Werten kann dann der rechnerisch bestimmte Pseudo-Zeitschritt

$$[\tau_{\text{opt}}] = \frac{|[\nu]|}{[L] \|\Delta x(\tau)\|} \geq \bar{\tau}_{\text{opt}} \geq \tau_{\text{opt}}$$

berechnet werden.

Falls keine Reduktion des Residuums vorliegt, $\|\Delta x(\tau)\| \geq \|F(x_0)\|$, ist laut (2.2.5) $\nu \geq [\nu] \geq 0$. In diesem Fall wird die Iteration abgebrochen werden, oder mit einer heuristischen Strategie weiter verfahren.

Ist die Bedingung $\nu < 0$ hingegen erfüllt, ist wie in (2.2.4) durch

$$[\nu]\tau = \tilde{\nu}(\tau)\tau = \tau \frac{(\Delta x, A\Delta x)}{\|\Delta x\|^2} = \frac{(\Delta x, \Delta x - F(x_0))}{\|\Delta x\|^2} \leq \nu\tau$$

eine gute Näherung für ν gegeben.

Für die Bestimmung eines geeigneten Wertes von L ergibt sich durch Umstellen der Terme im Beweis von Satz 2.2

$$\begin{aligned} \|F(x(\tau)) - \Delta x(\tau)\| &\leq \int_0^\tau \|(F'(x(\sigma)) - F'(x_0))(I - \sigma A)^{-2} F(x_0)\| d\sigma \\ &\leq L \int_0^\tau \|x(\sigma) - x_0\| \|(I - \sigma A)^{-1} \Delta x(\sigma)\| d\sigma. \end{aligned}$$

Das Integral lässt sich mit Hilfe der Trapez-Regel abschätzen durch

$$\begin{aligned}\|F(x(\tau)) - \Delta x(\tau)\| &\leq \frac{1}{2} L \|\Delta x(\tau)\|^2 \tau^2 / (1 - \nu\tau) + \mathcal{O}(\tau^4) \\ &\leq \frac{1}{2} L \|\Delta x(\tau)\|^2 \tau^2 + \mathcal{O}(\tau^4).\end{aligned}$$

Mit (2.2.5) ergibt sich als obere Grenze hierfür

$$\|F(x(\tau)) - \Delta x(\tau)\| \leq \frac{1}{2} L \|F(x_0)\|^2 \tau^2 / (1 - \nu\tau)^2,$$

was die Ableitung (2.2.12) aus dem Beweis von Satz 2.2 ist und durch Umstellen ergibt sich

$$[L] := \frac{2\|F(x(\tau)) - \Delta x\|}{\tau^2 \|\Delta x\|^2} \leq L + \mathcal{O}(\tau^2)$$

als geeignete Näherung für L .

Die beiden Werte für die Lipschitz-Bedingungen in die Formel für $[\tau_{\text{opt}}]$ eingesetzt, ergeben mit

$$[\tau_{\text{opt}}] = \frac{|(\Delta x(\tau), F(x_0) - \Delta x(\tau))|}{2\|\Delta x(\tau)\| \|F(x(\tau)) - \Delta x(\tau)\|} \tau \quad (2.3.3)$$

eine geeignete Näherung für die Berechnung der Zeitschrittweite.

Bemerkung 2.10. Mit einer Formel für die Berechnung der Zeitschrittweite gibt es eine adaptive Strategie zur Bestimmung der Zeitschrittweite mit zwei verschiedenen Ansätzen. Wenn das Residuum sich im Schritt von x_0 zu $x(\tau)$ nicht verringert, wird die Zeitschrittweite τ durch $[\tau_{\text{opt}}] < \tau$ korrigiert. Andernfalls, wenn sich das Residuum von x_0 nach $x(\tau)$ verringert, wird die nächste Iteration mit dem Probewert $[\tau_{\text{opt}}]$ gestartet.

Alternative Strategien zur Berechnung der Zeitschrittweite

Bisher gezeigt wurde die Berechnung der theoretisch optimalen Zeitschrittweite τ_{opt} . In der Praxis jedoch werden häufig heuristische Methoden zur Berechnung der Zeitschrittweite verwendet, die mit weniger Informationen oder mit weniger Zeit- und Rechenaufwand eine Näherung liefern. Ein Beispiel dafür ist die Switched Evolution Relaxation (SER), deren Ansatz aus dem Paper [KK98] von Kelley und Keyes stammt.

Die Grundliegende Idee hinter SER ist die Zeitschrittweite invers proportional zur Reduktion des Residuums zu erhöhen. Nach [KK98] folgt aus (2.3.2)

$$\tau_k = \tau_0 \frac{\|F(x_0)\|}{\|F(x_k)\|}.$$

Damit gilt für $k \geq 1$, dass

$$\tau_{k+1} = \tau_k \frac{\|F(x_k)\|}{\|F(x_{k+1})\|} \quad (2.3.4)$$

ein geeignete Näherung für die Zeitschrittweite ist.

Eine weitere Alternative, welche auch bei der späteren Anwendung des Algorithmus Einsatz gefunden hat, ist auf eine Reduktion des Residuums zu verzichten. Bei bestimmten Problemen ergibt sich für manche Punkten ein Problem mit der Berechnung der Zeitschrittweite, so dass diese abnimmt anstatt sich zu erhöhen. In einem solchen Fall nimmt auch das Residuum nicht ab und die Iteration wird abgebrochen werden. Erlaubt man allerdings, für Iterationen im transienten Bereich fern der Lösung, diese Abbruchbedingung zu ignorieren, führt dies häufig dazu, dass das Verfahren bis zu einer guten Approximation der Lösung weiter läuft. Für eine Anwendung dieser Strategie siehe das Switched Evolution Relaxation Modell in Abschnitt 3.4.2.

3. PTC Algorithmus und seine Anwendung

In diesem Kapitel wird der PTC Algorithmus vorgestellt. Beginnend mit seinem Pseudo-Code, anhand dessen die grundlegenden Abläufe klar gemacht werden. Danach wird der in Python implementierte Code vorgestellt und die verwendeten Methoden einzeln betrachtet.

Anschließend wird gezeigt wie man aus einem chemischen System, welches durch Reaktionsgleichungen beschrieben wird, ein mathematisches Modell erhält. Dieses Modell ist dann von einer Form welche der Algorithmus verwenden kann, um die Vorgänge des zugrundeliegenden Systems zu simulieren.

Im darauffolgenden Abschnitt befinden sich drei Beispiele zu deren Lösung der Algorithmus verwendet wurde. In den Beispielen wird jeweils ein System von seinem Anfangszustand in seinen Gleichgewichtszustand überführt. Die resultierenden Ergebnisse werden analysiert und für die ersten beiden Beispiele die Durchführung von zwei verschiedenen Strategien zur Bestimmung der Zeitschrittweite verglichen.

3.1. Vorstellung des Algorithmus

Der PTC Algorithmus folgt aus dem in Kapitel 2 vorgestellten Verfahren von [Deu11]. Als Grundlage für die spätere Implementierung dient der folgende Pseudo-Code.

Algorithm 1 Pseudo Transient Continuation

```
Setze  $x = x_0$  und  $\tau = \tau_0$ .  
Werte  $\|F(x)\|$  aus.  
while  $i < \text{max\_iter}$  und  $\|F(x)\| > \epsilon$  do  
    Löse  $(I - \tau F'(x))\Delta x = F(x)$ .  
    Setze  $x \leftarrow x + \tau \Delta x$ .  
    Werte  $\|F(x)\|$  aus.  
    if  $\|F(x)\|$  abgenommen then  
        Bestimme neue Zeitschrittweite  $\tau$ .  
         $i \leftarrow i + 1$   
    else  
        Beende die Iteration oder verfahre weiter mit heuristischer Strategie.  
end
```

Der Algorithmus beginnt mit der Übergabe von zwei Startwerten. Der anfängliche Zustand des Systems x_0 , in dem sämtliche Konzentrationen der Stoffe enthalten sind, sowie eine anfängliche Zeitschrittweite τ_0 , die als Grundlage für die nächsten Iterationen dient. Für den Zustand x_0 wird zunächst das Residuum ausgewertet. Liegt das Residuum über einer festgelegten unteren Schranke ϵ und sind noch nicht die maximale Anzahl an Iterationen max_iter durchlaufen worden, beginnt der Algorithmus damit den nächsten Zustand zu berechnen der ein kleineres Residuum besitzen soll.

Dafür wird im ersten Schritt jeder Iteration das lineare System $(I - \tau F'(x))\Delta x = F(x)$ nach Δx gelöst. Anschließend wird mit Δx ein Probewert für den Zustand des Systems in dieser Iteration berechnet. Für diesen wird getestet, ob das Residuum im Vergleich zur vorherigen Iteration abgenommen hat.

Wenn das Residuum abgenommen hat, wird der derzeitige Wert für den Zustand übernommen und der Algorithmus verfährt normal weiter, indem er die Zeitschrittweite berechnet die in der nächsten Iteration verwendet wird. Nimmt das Residuum hingegen nicht ab, wird der Algorithmus entweder beendet und der vorherige Zustand als Lösung zurückgegeben, oder mit einer heuristischen Strategie weiter verfahren. Der Algorithmus endet also entweder bei Erreichen eines Residuums $< \epsilon$, nach einer Iteration in der keine weitere Reduktion des Residuums erreicht wurde, oder nach Erreichen der maximalen Anzahl an Iterationen.

3.2. Implementierung in Python

In diesem Abschnitt werden die einzelnen Komponenten im Quellcode erklärt, welche zur Ausführung des PTC Algorithmus benötigt werden. Beginnend mit den verwendeten Methoden für die Berechnung der Änderungsrate, den Methoden zur Bestimmung der Zeitschrittweiten und zu Letzt vom Algorithmus. Der komplette Quellcode für den Algorithmus und alle Beispiele ist im Anhang in Kapitel B hinterlegt.

```
1 # Berechne die Änderungsrate Delta x
2 def delta_x(k, t, x):
3     i = np.eye(len(x))
4     dx = la.solve(i - t * jacob_i(k, x), fkt_wert(k, x))
5     return dx
```

Im ersten Schritt wird eine Methode zum Lösen des linearen Systems $F(x) = (I - \tau F'(x))\Delta x$ nach Δx benötigt. Das wird von der Methode `delta_x`, durch Verwendung von `solve()`, mit Hilfe von exakten Lösungsmethoden erreicht. `delta_x` benötigt als Eingabeparameter einen Vektor `k` mit benutzerdefinierten Daten wie z.B. Reaktionskonstanten, eine Zeitschrittweite `t` und einen aktuellen Zustand `x`. Es wird der Wert `dx` für Δx zurückgegeben.

```
1 # Berechne optimalen Zeitschritt
2 def t_opt(k, t_n, x_n, x_n1, dx):
3     z = abs(np.dot(dx, fkt_wert(k, x_n) - dx))
4     n = 2 * la.norm(dx) * la.norm(fkt_wert(k, x_n1) - dx)
5     return (z / n) * t_n
```

Als nächstes folgen die Methoden für die Bestimmung der Zeitschrittweite. Die erste Methode `t_opt` berechnet eine Zeitschrittweite mit der optimalen Strategie aus Abschnitt 2.3. Sie benötigt als Eingabeparameter den Vektor `k` mit verschiedenen benutzerdefinierten Daten, eine Zeitschrittweite `t_n`, die beiden Zustände `x_n` und `x_n1`, sowie die Änderungsrate `dx`. Sie basiert auf der Formel für die optimale Zeitschrittweite (2.3.3). Mit den übergebenen Werten gibt `t_opt` die errechnete optimale Zeitschrittweite `t_n1` zurück.

```
1 # Berechne SER Zeitschritt
2 def t_ser(k, t_n, x_n, x_n1):
3     z = t_n * la.norm(fkt_wert(k, x_n))
4     n = la.norm(fkt_wert(k, x_n1))
5     return z / n
```

Wie in Abschnitt 2.3 besprochen, werden in der Praxis häufig heuristische Methoden zur Bestimmung der Zeitschrittweite verwendet. Diese liefern eine Näherung, welche immer noch gut genug ist und benötigen entweder weniger Übergabeparameter, oder haben eine geringere Laufzeit. Die Beispiele 3.4.1 und 3.4.2 werden zusätzlich zu der Methode `t_opt` mit einer heuristischen

Strategie in der Methode `t_ser` berechnet. Diese benötigt dieselben Parameter, abgesehen von der Änderungsrate `dx`. Sie basiert auf der Switched Evolution Relaxation (2.3.4), die in Abschnitt 2.3 vorgestellt wird. Die Methode `t_ser` gibt ebenfalls die errechnete Zeitschrittweite `t_n1` zurück.

Mit Hilfe der gezeigten Methoden wird sodann der PTC Algorithmus implementiert.

```

1 # Berechne Beispiel
2 def calc_bsp(k, x0, t_n, stoffe, max_iter, opt, eps):
3     x_n = x0
4     x_list = [x0]
5     t_list = [t_n]
6     res_list = [la.norm(fkt_wert(k, x0))]
7     i=0
8     while i in range(0, max_iter) and la.norm(fkt_wert(k, x_n)) > eps:
9         dx = delta_x(k, t_n, x_n)
10        x_n1 = x_n + t_n * dx
11        if la.norm(fkt_wert(k, x_n1)) >= la.norm(fkt_wert(k, x_n)):
12            print('Abbruch: Keine Residual-Reduktion')
13            break
14        if opt:
15            t_n1 = t_opt(k, t_n, x_n, x_n1, dx)
16        else:
17            t_n1 = t_ser(k, t_n, x_n, x_n1)
18            t_list = np.vstack((t_list, t_n1))
19            res_list = np.vstack((res_list, la.norm(fkt_wert(k, x_n1))))
20            x_list = np.vstack((x_list, x_n1))
21            t_n = t_n1
22            x_n = x_n1
23            i = i + 1
24        df = pd.DataFrame(data=x_list, columns=stoffe)
25        df['Residuum'] = res_list
26        df['Zeitschrittweite'] = t_list
27        df.index.name = 'Iterationen'
28
29    return df

```

Der Algorithmus, in der Version zur Anwendung auf chemische Probleme, bekommt zu Anfangs ein Tupel übergeben. Dieses enthält benutzerdefinierten Daten im Vektor `k`, den anfänglichen Zustand des Systems im Vektor `x0`, die gewählte anfängliche Zeitschrittweite `t_n`, eine geordnete Liste mit den Namen der Reaktanten `stoffe`, die maximale Anzahl der durchführbaren Iterationen `max_iter`, einen Boolean Wert `opt` der festlegt welche Methode zur Bestimmung der Zeitschrittweite benutzt werden soll und einen Wert `eps` als untere Schranke für das Residuum.

Als Erstes werden die übergebenen Werte als aktuelle Parameter des Systems übernommen und Listen für die Zustände, Zeitschrittweiten und die Residuen der einzelnen Iterationen angelegt. Falls für `x_n` eine weitere Verbesserung des Residuums erreicht werden soll und noch nicht die maximale Anzahl an Iterationen `max_iter` durchlaufen worden ist, beginnt der Algorithmus mit Hilfe der `delta_x` Methode das `dx` zu bestimmen. Mit `dx` kann anschließend der neue Zustand `x_n1` berechnet werden.

Mit diesen Information wird jetzt überprüft, ob das Residuum in diesem Schritt abgenommen hat. Falls es nicht abgenommen hat, bricht der Algorithmus ab und übergibt als Lösung den derzeitigen Zustand für `x_n`. In manchen der betrachteten Beispielen wird stattdessen weiter

Verfahren ohne eine Reduktion des Residuums vorauszusetzen. Eine aus Abschnitt 2.3 bekannte heuristische Strategie. In diesen Fällen wird die Abbruchbedingung folgendermaßen modifiziert.

```

1      #Modifizierte Abbruchbedingung, verzichtet auf eine Reduktion des Residuums
2      in den ersten Iterationen
3      if la.norm(fkt_wert(args, x_n1)) >= la.norm(fkt_wert(args, x_n)):
4          if i < 3:
5              res_list = np.vstack((res_list, la.norm(fkt_wert(args, x_n1))))
6              x_list = np.vstack((x_list, x_n1))
7              if opt:
8                  t_n1 = t_opt(args, t_n, x_n, x_n1, dx)
9              else:
10                 t_n1 = t_ser(args, t_n, x_n, x_n1)
11                 t_list = np.vstack((t_list, t_n1))
12                 x_n = x_n1
13                 t_n = t_n1
14             else:
15                 print('Abbruch: Keine Residual-Reduktion')
16                 break

```

Reduziert sich das Residuum wie vorhergesehen, legt die boolesche Variable `opt` fest welche Methode zur Berechnung der Zeitschrittweite benutzt wird. Mit dieser berechnet der Algorithmus unter Verwendung der bisher berechneten Werte die nächste Zeitschrittweite `t_n1`. Sobald `t_opt` oder `t_ser` diese übergeben haben, wird `t_n1`, der Zustand `x_n1`, sowie das Residuum in die geeigneten Listen geschrieben und als aktuelle Parameter des Systems übernommen.

3.3. Mathematische Modellierung chemischer Systeme

Die Probleme deren Lösung in dieser Arbeit behandelt werden basieren auf chemischen Systemen, innerhalb deren Stoffe miteinander reagieren. Der quantitative Austausch von Stoffmengen der dabei auftritt wird in Reaktionsgleichungen beschrieben. Ziel dieses Abschnittes ist zu zeigen, wie aus einem solchem System ein mathematisches Modell konstruiert werden kann. Als praktisches Beispiel dienen Chromatographische Prozesse, mit denen in der Chemie unter anderem die stoffliche Reinheit von Gemischen sichergestellt wird.

In einem chemischen System sind alle Reaktionen und Beziehungen zwischen denen im System befindlichen Molekülen enthalten. Alle äußeren Einflüsse, die nicht zum System gehören, bilden dessen Umgebung. Ein solches System wird geschlossen genannt, wenn es keinen Austausch von Masse oder Energie mit seiner Umgebung betreibt. Alle Reaktionen werden in Reaktionsgleichungen beschrieben.

Das System soll von seinem anfänglichen Zustand in sein chemisches Gleichgewicht überführt werden. Das chemische Gleichgewicht beschreibt einen Zustand bei dem die Hin- und Rückreaktionen mit gleicher Geschwindigkeit ablaufen und sich somit die Mengen auf der Seite der Edukte und der Produkte nicht weiter ändern. Anders ausgedrückt sind die Flüsse von Hin- und Rückrichtung der Reaktionen gleich.

Ein solches System mit mehreren Reaktionsgleichungen fassen wir mathematisch als System gewöhnlicher Differentialgleichungen auf, dessen Überführung in seinen Gleichgewichtszustand der Überführung des chemischen Systems in sein chemisches Gleichgewicht entspricht. Um aus den Reaktionsgleichungen ein mathematisches Modell zu erzeugen, welches der Algorithmus verstehen kann, werden Hilfsmittel aus der Chemie benötigt. Zunächst ein Fundamentales Gesetz aus der Chemie, dessen gelten wir für alle folgenden Aussagen voraussetzen.

Satz 3.1. (Massenwirkungsgesetz)

In einem geschlossenem System besitzt, bei einer reversiblen Reaktion im chemischen Gleichgewicht, der Quotient aus Produkt der Reaktanten und dem Produkt der Ausgangsstoffe einen festen Wert K . Dieser Wert K wird Gleichgewichtskonstante der Reaktion genannt und besitzt einen festen, für die Reaktion charakteristischen Wert.

Anders ausgedrückt besagt das Massenwirkungsgesetz, dass sich in einem geschlossenen System die Gesamtmenge der Moleküle nie verändert, also ist der Fluss einer chemischen Reaktion proportional zum Produkt der Konzentrationen ihrer Reaktanten. Als Konsequenz daraus bilden Edukte und Produkte einer Reaktion eine Erhaltungsgröße, die bei der Konstruktion des Modells beachtet werden muss.

Die Änderung von Stoffmengen innerhalb eines geschlossenen Systems wird mit Hilfe der stöchiometrischen Rechnung bestimmt. Sie beinhaltet die Änderung aller Stoffmengen von Edukten und Produkten einer Reaktion und wird insgesamt mit der Stöchiometrie des Systems bezeichnet. Die stöchiometrische Matrix S stellt die Stöchiometrie kompakt dar. S zu einem System mit m Reaktanten und n Reaktionen ist eine Matrix $S \in \mathbb{Z}^{m \times n}$. Die Einträge $S_{i,j}$ drücken die Menge des Reaktant i aus, welche in Reaktion j verbraucht wurden oder entstanden ist. Die Stöchiometrie ist Grundlagenwissen in der Chemie, weiterführende Informationen zum Thema finden sich z.B. in [Hil09]. Während man in der Chemie häufig die Änderung von Stoffmengen betrachtet, arbeiten wir mit den Konzentrationen der Stoffe. Alle getroffenen Aussagen über Stoffmengen gelten äquivalent für die zugehörigen Konzentrationen.

Der Fluss beschreibt die Rate, mit der die einzelnen Reaktanten reagieren. Er wird auch als Reaktionsgeschwindigkeit bezeichnet. Jede Reaktionsgleichung besitzt ihren eigenen Fluss. Der gesamte Fluss eines Systems ist ein Vektor v , der alle Flüsse der einzelnen Reaktionsgleichungen als Einträge hat.

Mit diesem Vorwissen können, wenn die Reaktionsgleichungen bekannt sind, die stöchiometrische Matrix S sowie der Fluss $v(C)$ des Systems gebildet werden. Sie beschreiben dann das System gewöhnlicher Differentialgleichungen

$$\frac{d}{dt}C(t) = \dot{C}(t) = Sv(C). \quad (3.3.1)$$

Bemerkung 3.1. Wir beobachten, dass die Veränderungen der Konzentration über die Zeit $\dot{C}(t)$ komponentenweise addiert werden können.

Sei $\dot{C}_i = S_i v(C)$ die i -te Zeile des Systems (3.3.1), mit $i = 1, \dots, n$ und seien i, j , mit $i \neq j$ zwei beliebige Reaktanten aus dem System. Dann lassen sich die zugehörigen Zeilen im System wie folgt kombinieren

$$\dot{C}_i(t) + \dot{C}_j(t) = (S_i + S_j)v. \quad (3.3.2)$$

Bilden also zwei Reaktanten eine Erhaltungsgröße, ist die Summe der Änderungen ihrer Konzentrationen über die Zeit gleich null. Laut 3.3.2 ist in diesem Fall auch die Summe der korrespondierenden Zeilen in der stöchiometrischen Matrix ebenfalls gleich null.

3.4. Beispiele und Ergebnisse

In diesem Abschnitt wird der in Abschnitt 3.2 vorgestellte Algorithmus zum Lösen verschiedener Beispiele verwendet und die Ergebnisse analysiert. Für jedes Beispiel werden eine Herleitung des zu lösenden Systems gewöhnlicher Differentialgleichungen und ein kurzer Abriss der chemischen Hintergründe gegeben. Außerdem wird das Modell und die zugehörige Jacobi-Matrix in Python implementiert. Dabei wird auf eventuell verwendete heuristische Strategien hingewiesen.

Das erste Beispiel ist ein einfaches System mit drei Reaktanten und soll das im vorherigen Abschnitt vermittelte Konzept klarer werden lassen. Die darauf folgenden Beispiele lösen echte Probleme aus der Chemie, deren Parameter mir von Samuel Lewke [Lew] zur Verfügung gestellt worden sind.

3.4.1. Beispiel 1 – Einführendes Beispiel

In diesem Beispiel betrachten wir ein System mit drei Stoffen und einer reversiblen Reaktion. Wie in 3.3 erhalten wir aus der Reaktionsgleichung den Fluss und alle Informationen zur Bildung der stöchiometrischen Matrix, mit denen wir das mathematische Modell konstruieren können.

Das chemische Problem (mit den drei Reaktanten A, B und C) sei gegeben durch:



Mit dem Massenwirkungsgesetz ergeben sich die stöchiometrische Matrix und der Fluss

$$S = \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix}, \quad v(C) = \left(k_1 C_a C_b - k_2 C_c \right).$$

Bei der Hinreaktion ($\rightarrow$) reagieren jeweils ein A und ein B und bilden ein C. Das korrespondiert mit den Einträgen in der Spalte von S . Da nur eine Reaktionsgleichung existiert hat S die Form eines Vektors. Der Fluss setzt sich aus den Konzentrationen der einzelnen Reaktanten zusammen und die $k_{1,2}$ sind die Reaktionskonstanten der Hin- und Rückreaktion. Wie besprochen bilden Edukt und Produkt jeweils eine Erhaltungsgröße, also ergeben sich hier

$$E_1 = C_a + C_c \text{ und} \\ E_2 = C_b + C_c.$$

Wie in (3.3.1) bilden diese beiden das System gewöhnlicher Differentialgleichungen

$$\dot{C} = \begin{pmatrix} \dot{C}_a \\ \dot{C}_b \\ \dot{C}_c \end{pmatrix} = \begin{pmatrix} -k_1 C_a C_b + k_2 C_c \\ -k_1 C_a C_b + k_2 C_c \\ k_1 C_a C_b - k_2 C_c \end{pmatrix}.$$

Dann wird $\dot{C}$ wie folgt in Python implementiert:

```
1 # Berechne Funktionswerte
2 def fkt_wert(k, x):
3     return np.array(
4         [-k[0] * x[0] * x[1] + k[1] * x[2],
5          -k[0] * x[0] * x[1] + k[1] * x[2],
6          k[0] * x[0] * x[1] - k[1] * x[2]])
```

Durch Ableiten von $\dot{C}$ erhalten wir die zugehörige Jacobi-Matrix:

```

1 # Berechne Jacobi-Matrix
2 def jacobi(k, x):
3     return np.array(
4         [[-k[0] * x[1], -k[0] * x[0], k[1]],
5          [-k[0] * x[1], -k[0] * x[0], k[1]],
6          [k[0] * x[1], k[0] * x[0], -k[1]]])

```

Eingabeparameter

Der Algorithmus benötigt den anfänglichen Zustand des Systems C_0 und eine Startzeitschrittweite t_0 . Wir lassen die Iteration mit den anfänglichen Konzentrationen

$$C_0 = (0.25, 1.25, 0.75)$$

starten und legen die erste Zeitschrittweite mit $t_0 = 1$ fest.

Weitere benötigte Parameter sind die maximale Anzahl an Iterationen $\text{max_iter} = 50$, einen Vektor $k = (0.2, 0.3)^T$ mit den Reaktionskonstanten und die untere Schranke für das Residuum $\text{eps} = 1 * 10^{-10}$.

Verwendung der optimalen Pseudo-Zeitschrittweite

Es wird die Strategie zur Berechnung der optimalen Pseudo-Zeitschrittweite (OPT) aus Abschnitt 2.3 verwendet. Der Algorithmus terminiert entweder nach Erreichen der maximalen Anzahl an Iterationen max_iter , wenn in einer Iteration keine Reduktion des Residuums mehr stattfindet, oder das Residuum in der vorherigen Iteration einen Wert kleiner eps erreicht hat.

Nach seiner Terminierung gibt der Algorithmus einen DataFrame aus, der Aufschluss über die Entwicklung der Konzentrationen, der Residuen und der Zeitschrittweiten gibt. Zusätzlich wird auf der Konsole der Grund für das Terminieren ausgegeben.

Iterationen	C_a	C_b	C_c	Residuum	Zeitschrittweite
0	0.250	1.250	0.750	0.281	1.000
1	0.352	1.352	0.648	0.172	14.769
2	0.492	1.492	0.508	$9.638 \cdot 10^{-3}$	168.391
3	0.500	1.500	0.500	$5.973 \cdot 10^{-5}$	37,045.945
4	0.500	1.500	0.500	$1.463 \cdot 10^{-9}$	$1.316 \cdot 10^9$
5	0.500	1.500	0.500	$4.807 \cdot 10^{-17}$	$1.938 \cdot 10^{16}$

Tabelle 1: Ergebnisse aus Beispiel 1 unter Verwendung von OPT

Wie aus Tabelle 1 ersichtlich, terminiert der Algorithmus nach der fünften Iteration. Aus der letzten Zeile lässt sich die berechnete Lösung

$$C_{\text{OPT}} = (0.5, 1.5, 0.5).$$

ablesen. Das System hat am errechneten Zustand ein Residuum von ca. $4.8 * 10^{-17}$. Mit diesem Wert ist der Algorithmus erfolgreich terminiert und hat eine sehr gute Näherung an den

Gleichgewichtspunkt des Systems gefunden. Der Algorithmus ist terminiert, weil in der fünften Iteration ein Residuums $< \mathbf{eps}$ erreicht wird.

Wie zu erwarten wird die Zeitschrittweite in jedem Iterationsschritt exponentiell erhöht ($\rightarrow \infty$) und das Residuum verläuft invers proportional ($\rightarrow 0$).

Bestimmung der Pseudo-Zeitschrittweite mit Switched Evolution Relaxation

Die Iteration wird mit denselben anfänglichen Konzentrationen C_0 gestartet, aber anstatt OPT wird Switched Evolution Relaxation (SER) zur Berechnung der Zeitschrittweiten verwendet. Es werden die selben Abbruchbedingungen wie zuvor verwendet.

Iterationen	C_A	C_B	C_C	Residuum	Zeitschrittweite
0	0.250	1.250	0.750	0.281	1.000
1	0.352	1.352	0.648	0.172	1.633
2	0.431	1.431	0.569	$8.204 \cdot 10^{-2}$	3.431
3	0.480	1.480	0.520	$2.397 \cdot 10^{-2}$	11.741
4	0.498	1.498	0.502	$2.517 \cdot 10^{-3}$	111.824
5	0.500	1.500	0.500	$3.033 \cdot 10^{-5}$	9,280.623
6	0.500	1.500	0.500	$4.451 \cdot 10^{-9}$	$6.323 \cdot 10^7$
7	0.500	1.500	0.500	$1.442 \cdot 10^{-16}$	$1.952 \cdot 10^{15}$

Tabelle 2: Ergebnisse aus Beispiel 1 unter Verwendung von SER

Wie aus Tabelle 2 ersichtlich terminiert der Algorithmus nach der siebten Iteration und gibt eine berechnete Lösung mit

$$C_{\text{SER}} = (0.5, 1.5, 0.5),$$

an. Mit einem Residuum von ca. $1.4 \cdot 10^{-16}$ ist er erfolgreich terminiert und hat ebenfalls eine sehr gute Approximation des Gleichgewichtspunkts geliefert. Wie bei OPT hat er terminiert, weil in der siebten Iteration ein Residuum $< \mathbf{eps}$ erreicht wird und die Verläufe der Residuen und Zeitschrittweiten stimmen mit den erwarteten Werten überein.

Vergleich der beiden Zeitschrittweiten für Beispiel 1

Die beiden Abbildungen 1 und 2 beschreiben für die jeweiligen Zeitschrittweiten den Verlauf der Konzentrationen und der genormten Residuen. Bei diesem Beispiel bringt OPT das System in weniger Iterationen in seinen Gleichgewichtszustand. Das korrespondiert mit den Werten für die Zeitschrittweiten. OPT erhöht diese im Vergleich mit SER, gerade in den ersten Iterationen, in deutlich größeren Schritten. Dementsprechend verringert sich das Residuum schneller und der Algorithmus terminiert in weniger Iterationen.

Das wesentliche Ergebnis dieses einführenden Beispiels ist, dass sich beide Strategien gut eignen, Probleme dieser Art zu lösen. Trotz der ähnlichen Ergebnisse ($C_{\text{OPT}} \approx C_{\text{SER}}$) lässt sich an den beiden Graphen 1 und 2 noch einmal sehen, dass einen schnellere Erhöhung der Zeitschrittweiten zu einer schnelleren Reduktion des Residuums führt.

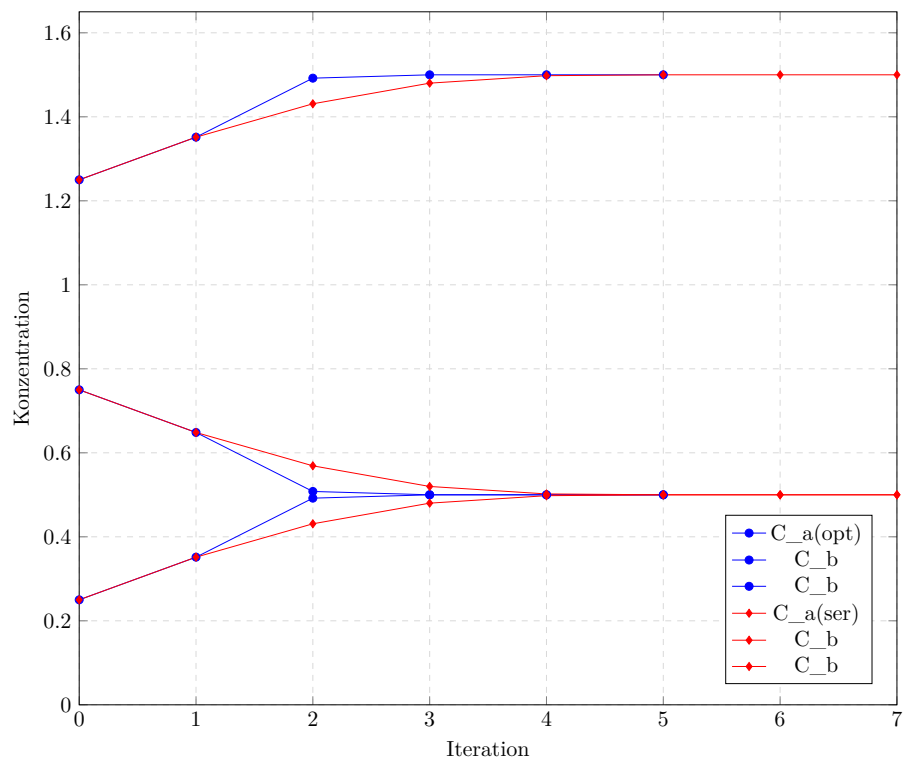


Abbildung 1: Konzentrationen der Reaktanten in Beispiel 1

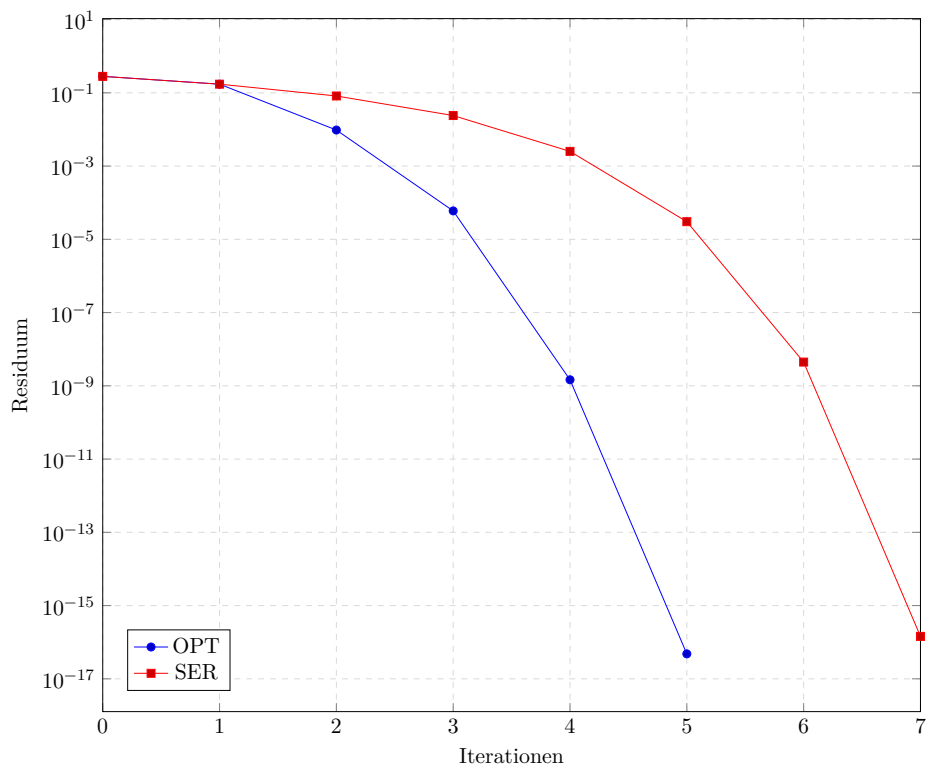


Abbildung 2: Verlauf der Residuen in Beispiel 1

3.4.2. Beispiel 2 – Steric Mass-Action Modell

Steric Mass-Action (SMA) ist ein Modell, das die Bindung von in einer Flüssigkeit gelösten Molekülen an eine feste Oberfläche beschreibt. Auf der Oberfläche befinden sich eine endliche Anzahl Λ von Bindungsstellen, an welche sich die Moleküle binden können. Die Bindung erfolgt durch die unterschiedliche elektrische Ladung von Oberfläche und Molekül. In der Flüssigkeit und auf der Oberfläche befinden sich zusätzlich Gegenionen, die an den Bindungsstellen elektrische Neutralität herstellen. Wenn ein Molekül an die Oberfläche adsorbiert, verdrängt es die dort befindlichen Gegenionen, die wieder in die Flüssigkeit freigesetzt werden. Umgekehrt desorbieren an der Oberfläche gebundene Moleküle zurück in die Flüssigkeit, wenn die Konzentration der Salz-Ionen in dieser groß genug ist. So lässt sich die Bindung der Moleküle durch die Steuerung der Gegenion-Konzentration kontrollieren. Die Anzahl der pro Molekül freigesetzten Gegenionen wird durch die charakteristische Ladung ν eines Proteins bestimmt. Verschiedene Moleküle besitzen verschiedene Ladungsverteilungen und haben somit eine unterschiedliche Bindungsaffinität zur Oberfläche.

Zusätzlich zu beachten ist, dass Proteine auf Grund ihrer physischen Ausdehnung Bindungsstellen verdecken können. Die Anzahl der Bindungsstellen, die von einem Protein verdeckt werden, bezeichnet man als sterischen Faktor σ des Proteins. Die dort befindlichen Bindungsstellen sind vom Protein „verdeckt“ und stehen nicht mehr für andere Proteine zur Verfügung. Die verdeckten Gegenionen heißen sterisch abgeschirmte Ionen und müssen bei der Erstellung eines Modells beachtet werden.

Die Moleküle in der Flüssigkeit werden über die Oberfläche bewegt und binden sich an verfügbare Bindungsstellen, wobei sich die dort befindlichen Gegenionen lösen. Dieser Prozess und seine Umkehrung findet zu jeder Zeit in hoher Frequenz statt. Ziel ist es, ein mathematisches Modell zu finden, welches dieses System beschreibt und dessen Gleichgewichtszustand zu bestimmen.

Vorüberlegungen

Die folgenden Beobachtungen und Hintergründe für die Erstellung eines Steric Mass-Action Modells stammen aus dem Journal von Brooks und Cramer [BC92]. Im betrachteten Prozess werden Salz-Ionen als Gegenionen auf der Oberfläche und Proteine als in der Trägerflüssigkeit gelöste Moleküle verwendet. Die Kapazität Λ ist die gesamte (endliche) Anzahl von Bindungsstellen auf der Oberfläche. Die Reaktionsgleichung für den oben vorgestellten Prozess mit nur einem Protein lautet



Dabei gilt folgende Notation: P ist ein freies Protein, gelöst in Flüssigkeit; FS ist eine freie Bindungsstelle auf der Oberfläche, gebunden mit einem Salz-Ion; S ist ein freies Salz-Ion in der Flüssigkeit; BP ist ein an die Oberfläche gebundenes Protein und ν die charakteristische Ladung des Proteins.

Bemerkung 3.2. Laut [BC92] erlaubt die Annahme, dass das System thermodynamisch ideal ist, den Prozess unter Verwendung von Konzentrationen, statt Stoffmengen, zu beschreiben. Zusätzlich wird angenommen, dass immer Elektroneutralität auf der Oberfläche gilt. Alle Aussagen werden zunächst für ein einfaches Modell mit einem Salz-Ion und einem Protein getroffen. Wir werden aber sehen, dass sich dies leicht auf ein Modell mit mehreren Proteinen erweitern lässt.

Wir schreiben die Reaktionsgleichung (3.4.1) also unter Verwendung von Konzentrationen auf und erhalten



Dabei steht C für die Konzentrationen in der Flüssigkeit und Q für die Konzentration auf der Oberfläche. Die Superskripte $()^s$ und $()^p$ unterscheiden jeweils ob es sich um ein Salz-Ion oder ein Protein handelt. Die Anzahl der auf der Oberfläche, zur Bindung mit dem Protein verfügbaren, Bindungsstellen ist definiert als $\bar{Q}^{(s)}$.

Die Konzentration der sterisch abgeschirmten Salz-Ionen auf der Oberfläche ist definiert als $\hat{Q}^{(s)} = \sigma^{(p)} Q^{(p)}$, mit $\sigma^{(p)}$ dem sterischen Faktor des Proteins und $Q^{(p)}$ der Konzentration des Proteins auf der Oberfläche. Anders ausgedrückt also die verdeckten Stellen pro Protein mal der Konzentration des Proteins. Die gesamte Konzentration an Salz-Ionen auf der Oberfläche ist dann offensichtlich

$$Q^{(s)} = \hat{Q}^{(s)} + \bar{Q}^{(s)}. \quad (3.4.2)$$

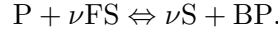
Sei $\nu^{(p)}$ die charakteristische Ladung des Proteins, dann drückt $\nu^{(p)} Q^{(p)}$ die Anzahl der durch gebundene Proteine belegten Bindungsstellen aus. Da nach Voraussetzung auf der Oberfläche immer Elektroneutralität herrscht, folgt für die Kapazität

$$\Lambda = Q^{(s)} + \nu^{(p)} Q^{(p)}. \quad (3.4.3)$$

Herleitung des Modells

Das Modell, sowie alle darin verwendeten Parameter, werden mir von Samuel Leweke [Lew] zur Verfügung gestellt. Betrachtet wird der Prozess unter Verwendung von einem Salz-Ion und drei verschiedenen Proteinen.

In einem System mit Proteinen wird der Austausch jedes einzelnen Proteins mit dem Salz-Ion getrennt in einer eigenen Gleichung betrachtet. Wir haben also für jedes der Proteine eine Reaktionsgleichung wie in 3.4.1



Daraus ergibt sich unter Verwendung des Massenwirkungsgesetz die stöchiometrische Matrix und der Fluss

$$S = \begin{pmatrix} \nu_1 & \nu_2 & \nu_3 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \\ -\nu_1 & -\nu_2 & -\nu_3 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad v = \begin{pmatrix} k_{a,1} C_1^{(p)} \left(\bar{Q}_0 \right)^{\nu_1} - k_{d,1} Q_1^{(p)} \left(C_0^{(p)} \right)^{\nu_1} \\ k_{a,2} C_2^{(p)} \left(\bar{Q}_0 \right)^{\nu_2} - k_{d,2} Q_2^{(p)} \left(C_0^{(p)} \right)^{\nu_2} \\ k_{a,3} C_3^{(p)} \left(\bar{Q}_0 \right)^{\nu_3} - k_{d,3} Q_3^{(p)} \left(C_0^{(p)} \right)^{\nu_3} \end{pmatrix}.$$

Nun kommt hierzu noch die Austauschreaktion der Gegenionen. Diese verläuft aber viel schneller als die Bindungsreaktionen der Proteine, so dass sie immer im Gleichgewicht ist. Die Gesamte Konzentration der Salz-Ionen $C_0^{(s)} + Q_0^{(s)}$ fassen wir als eine Erhaltungsgröße auf, deren Verhältnis in der Gleichung

$$\frac{d(C_0^{(s)} + Q_0^{(s)})}{dt} = 0 \quad (3.4.4)$$

festgesetzt wird. Das Verhältnis ist durch die Forderung der Elektroneutralität gegeben. Insgesamt erhalten wir das Modell

$$\frac{d}{dt} \begin{pmatrix} C_1^{(p)} \\ C_2^{(p)} \\ C_3^{(p)} \\ Q_1^{(p)} \\ Q_2^{(p)} \\ Q_3^{(p)} \end{pmatrix} = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} k_{a,1} C_1^{(p)} (\bar{Q}_0)^{\nu_1} - k_{d,1} Q_1^{(p)} (C_0^{(p)})^{\nu_1} \\ k_{a,2} C_2^{(p)} (\bar{Q}_0)^{\nu_2} - k_{d,2} Q_2^{(p)} (C_0^{(p)})^{\nu_2} \\ k_{a,3} C_3^{(p)} (\bar{Q}_0)^{\nu_3} - k_{d,3} Q_3^{(p)} (C_0^{(p)})^{\nu_3} \end{pmatrix}. \quad (3.4.5)$$

Wir setzen die Konzentrationen in der Flüssigkeit und die Erhaltungsgröße (3.4.4) als konstant fest und schauen welches Gleichgewicht sich auf der Oberfläche einstellen wird. Dann ergibt sich mit (3.4.3) auf der Oberfläche

$$Q_0^{(s)} = \Lambda - \sum_{j=1}^3 Q_j^{(p)} \nu_j$$

und somit

$$0 = \Lambda - \sum_{j=1}^3 Q_j^{(p)} \nu_j - Q_0^{(s)}$$

im Gleichgewicht. Von diesem Modell wollen wir den Gleichgewichtszustand (für gegebene $C_0^{(s)}, C_1^{(p)}, C_2^{(p)}, C_3^{(p)}$) bestimmen. Wir müssen also das Problem $F(Q) = 0$ lösen, wobei F die rechte Seite des obigen Systems (3.4.5) ist. Ein solches System kann zum Beispiel mit einem Newton-Verfahren gelöst werden.

Der Gleichgewichtszustand mit $F(Q) = 0$ ist aber auch Gleichgewichtszustand der folgenden ODE mit Pseudo-Zeit τ

$$\frac{d}{d\tau} \begin{pmatrix} Q_0^{(s)} \\ Q_1^{(p)} \\ Q_2^{(p)} \\ Q_3^{(p)} \end{pmatrix} = \begin{pmatrix} \Lambda - \sum_{j=1}^3 Q_j^{(p)} \nu_j - Q_0^{(s)} \\ k_{a,1} C_1^{(p)} (\bar{Q}_0)^{\nu_1} - k_{d,1} Q_1^{(p)} (C_0^{(p)})^{\nu_1} \\ k_{a,2} C_2^{(p)} (\bar{Q}_0)^{\nu_2} - k_{d,2} Q_2^{(p)} (C_0^{(p)})^{\nu_2} \\ k_{a,3} C_3^{(p)} (\bar{Q}_0)^{\nu_3} - k_{d,3} Q_3^{(p)} (C_0^{(p)})^{\nu_3} \end{pmatrix}. \quad (3.4.6)$$

Und ein System dieser Form können wir mit PTC lösen. Für den PTC Algorithmus ist das System (3.4.6) folgendermaßen von [Lew] in Python implementiert worden.

```

1 # Berechne Funktionswerte
2 def fkt_wert(args, x):
3     yCp = args[0]
4     kA = args[1]
5     kD = args[2]
6     nu = args[3]
7     sigma = args[4]
8     Lambda = args[5]
9     q0bar = x[0] - np.sum(sigma * x)
10    c0powNu = np.power(yCp[0], nu)
11    q0barPowNu = np.power(q0bar, nu)
12
13    fw = np.zeros(len(x))
14    fw[0] = -x[0] + Lambda - np.sum(nu * x)
15    fw[1:] = -kD[1:] * x[1:] * c0powNu[1:] + kA[1:] * yCp[1:] * q0barPowNu[1:]
16    return fw

```

Durch ableiten nach den einzelnen Reaktanten ergibt sich, ebenfalls von [Lew] implementiert, die folgende Methode für die zugehörige Jacobi-Matrix.

```

1 # Berechne Jacobi-Matrix
2 def jacobi(args, x):
3     yCp = args[0]
4     kA = args[1]
5     kD = args[2]
6     nu = args[3]
7     sigma = args[4]
8     # Lambda = args[5]
9
10    q0bar = x[0] - np.sum(sigma * x)
11    c0powNu = np.power(yCp[0], nu)
12    # q0barPowNu = np.power(q0bar, nu)
13    q0barPowNuM1 = np.power(q0bar, nu - 1.0)
14    extSigma = -sigma
15    extSigma[0] = 1.0
16
17    jm = np.zeros((len(x), len(x)))
18    jm[0, 0] = -1.0
19    jm[0, 1:] = -nu[1:]
20
21    for i in range(1, len(x)):
22        jm[i, :] = +kA[i] * yCp[i] * nu[i] * q0barPowNuM1[i] * extSigma
23        jm[i, i] -= kD[i] * c0powNu[i]

```

Eingabeparameter

Das Verfahren wird mit den anfänglichen Konzentrationen

$$C_0 = (700, 4, 14, 5)$$

gestartet. Der erste Eintrag beschreibt die Konzentration der Salz-Ionen und die anderen die der drei Proteine. Die erste Zeitschrittweite sei mit $t_0 = 1$ vorgegeben, die maximale Anzahl Iterationen mit `max_iter` = 50 und eine untere Schranke für das Residuum mit `eps` = $1 * 10^{-10}$. Alle zusätzlichen Modellparameter werden in einem Vektor `args` übergeben, der sich folgendermaßen zusammensetzt.

`args[0]`: yCp, Vektor mit den Konzentration der Proteine in der Flüssigkeit.
`args[1]`: kA, Vektor mit den Reaktionskonstanten der Hinreaktion.
`args[2]`: kD, Vektor mit den Reaktionskonstanten der Rückreaktion.
`args[3]`: nu, Vektor mit den charakteristischen Ladungen der Proteine.
`args[4]`: sigma, Vektor mit den sterischen Faktoren der Proteine.
`args[5]`: Lambda, Wert für die Kapazität der Oberfläche.

Bemerkung 3.3. Für manche Punkte nahe des Gleichgewichtspunkts `x_ref` werden in den ersten Iterationen des Verfahrens die Zeitschrittweiten nur sehr langsam erhöht oder sogar leicht verringert, so dass das Verfahren direkt in den ersten Iterationen abbricht. Um diesen Effekt vorzubeugen habe ich in der Implementierung von SMA dem Algorithmus erlaubt in diesen Iterationen ohne eine Reduktion des Residuums weiter zu verfahren, eine heuristisches Strategie aus Abschnitt 2.3. In diesem Fall hat dies die Erfolgsquote des Algorithmus deutlich erhöht. Die Auswirkungen lassen sich gut in Kapitel 4 sehen, siehe Bemerkung 4.2.

Verwendung der optimalen Pseudo-Zeitschrittweite (OPT)

Es wird die, aus Abschnitt 2.3 bekannte, Strategie zur Berechnung der optimalen Zeitschrittweite (OPT) verwendet. Der Algorithmus terminiert, weil er keine weitere Reduktion des Residuums festgestellt hat. Nach erfolgreichem terminieren gibt er einen DataFrame zurück der durch Tabelle 3 dargestellt wird. In dieser sind die Verläufe der Konzentrationen, Residuale und Zeitschrittweiten festgehalten.

Iterationen	C_a	C_b	C_c	C_d	Residuum	Zeitschrittweite
0	700.00000	4.00000	14.00000	5.00000	$2.814 \cdot 10^{13}$	1.000
1	910.34569	6.02424	5.07299	6.52934	$2.011 \cdot 10^{13}$	1.891
2	1,009.34818	10.60614	10.30144	9.17814	$8.919 \cdot 10^{11}$	4.791
3	1,043.19382	11.47610	11.32459	9.70177	$3.254 \cdot 10^{10}$	8.383
4	1,048.12824	11.59413	11.45768	9.77826	$1.11 \cdot 10^9$	14.045
5	1,048.55524	11.60418	11.46894	9.78487	$9.044 \cdot 10^6$	24.078
6	1,048.57783	11.60471	11.46952	9.78522	26,534.082	112.987
7	1,048.57854	11.60473	11.46954	9.78523	26.785	1,537.362
8	1,048.57855	11.60473	11.46954	9.78523	$3.937 \cdot 10^{-3}$	$1.436 \cdot 10^5$

Tabelle 3: Ergebnisse aus Beispiel 2 unter Verwendung von OPT

In der letzten Zeile des DataFrame lässt sich die approximierte Lösung

$$C_{\text{OPT}} = (1048.57855, 11.60473, 11.46954, 9.78523).$$

ablesen. An dieser ergibt sich für das Residuum ein Wert von ca. $4 \cdot 10^{-3}$. Wie zu erwarten wird die Zeitschrittweite in jedem Iterationsschritt exponentiell erhöht und geht gegen ∞ . Die Residuen verlaufen invers proportional dazu gegen 0.

Bestimmung der Pseudo-Zeitschrittweite mit Switched Evolution Relaxation (SER)

Es wird die, ebenfalls aus Abschnitt 2.3 bekannte, heuristische Strategie Switched Evolution Relaxation zur Berechnung der Zeitschrittweiten verwendet. Der Algorithmus gibt, nachdem er terminiert, den folgenden DataFrame aus.

Iterationen	C_a	C_b	C_c	C_d	Residuum	Zeitschrittweite
0	700.00000	4.00000	14.00000	5.00000	$2.814 \cdot 10^{13}$	1.000
1	910.34569	6.02424	5.07299	6.52934	$2.011 \cdot 10^{13}$	1.400
2	1,000.29901	10.38664	10.05145	9.03813	$1.007 \cdot 10^{12}$	27.935
3	1,047.36712	11.56891	11.42534	9.76596	$7.322 \cdot 10^{10}$	384.343
4	1,048.57615	11.60467	11.46948	9.78519	$9.5 \cdot 10^6$	$2.962 \cdot 10^6$
5	1,048.57855	11.60473	11.46954	9.78523	277.732	$1.013 \cdot 10^{11}$
6	1,048.57855	11.60473	11.46954	9.78523	$1.959 \cdot 10^{-2}$	$1.437 \cdot 10^{15}$

Tabelle 4: Ergebnisse aus Beispiel 2 unter Verwendung von SER

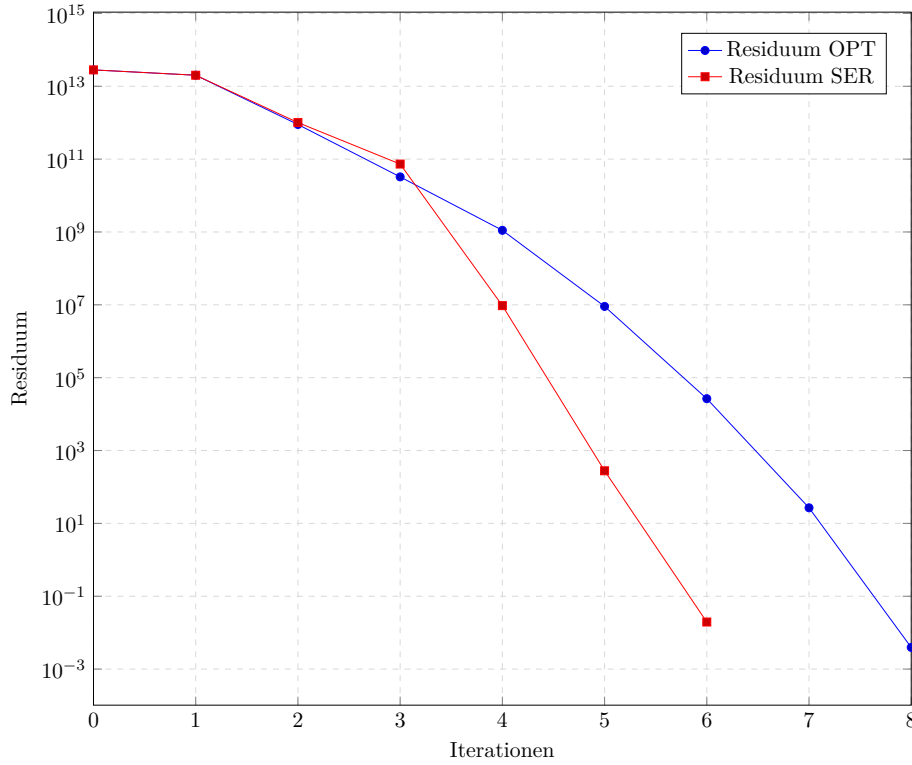


Abbildung 3: Norm des Residuums in Beispiel 2

Aus Tabelle 4 lässt sich ablesen, dass der Algorithmus nach der sechsten Iteration terminiert und die approximative Lösung

$$C_{\text{SER}} = (1048.57855, 11.60473, 11.46954, 9.78523),$$

mit einem Residuum von ca. 2×10^{-2} zurück gegeben hat. Auch hier hat der Algorithmus terminiert, weil in der siebten Iteration keine Reduktion des Residuums mehr vorliegt.

Vergleich der beiden Zeitschrittweiten für Beispiel 2

Abbildung 3 zeigt den Verlauf der Residuen beider Verfahren. Im Hinblick auf die Entwicklung der Zeitschrittweiten wird klar, dass SER diese für dieses Beispiel deutlich schneller und mehr erhöht. Dieser Effekt ist besonders gegen Ende des Verfahrens zu beobachten, wenn sich die Konzentrationen schon in einer Umgebung um die Lösung befinden. Dadurch konvergiert SER hier schneller als OPT gegen die Gleichgewichtslösung.

Bemerkung 3.4. Im Beispiel mit diesem Startzustand C_0 hat sich für das SMA Modell SER als die bessere Methode zur Bestimmung der Zeitschrittweite ergeben. Auch in Tests mit vielen verschiedenen Startwerten hat SER kontinuierlich besser abgeschnitten und wird aus diesem Grund für die Anwendung in Kapitel 4 verwendet, die sich auch mit dem SMA Modell beschäftigt.

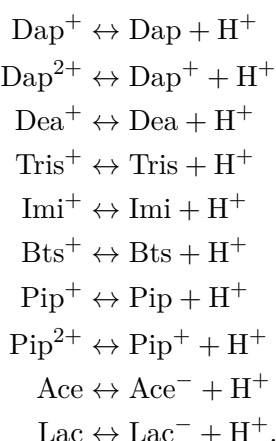
3.4.3. Beispiel 3 – Komplexes pH-Puffer Modell

Bei diesem Beispiel betrachten wir ein komplexes pH-Puffer Modell. Das Modell und die Modellparameter werden mir von Samuel Leweke [Lew] zur Verfügung gestellt.

Als Protonierung wird der Vorgang bezeichnet, bei dem einem Atom ein (positiv geladenes) Proton hinzugefügt wird. Umgekehrt heißt das Abgeben eines Protons Deprotonierung. Das abgebende Teilchen besitzt nach der Protonierung eine stärkere negative Ladung und wird mit einem - gekennzeichnet, oder verliert eines seiner +. Dieser Vorgang kann mehrmals auf dasselbe Teilchen angewandt werden, was die Ladung des entstehenden Ions immer negativer werden lässt.

Bemerkung 3.5. Ein Ion bezeichnet ein elektrisch geladenes Atom oder Teilchen. Die Ladung wird durch die Menge der Protonen und Elektronen bestimmt, die das Teilchen bekommen oder abgegeben hat.

Das Beispiel besteht aus einem System mit zehn Reaktionsgleichungen und neunzehn Reaktanten. Darunter befinden sich verschiedene Ladungszustände der Atome Dap, Dea, Tris, Imi, Bts, Pip, Ace und Lac, sowie einfach positive geladene Wasserstoffkerne H^+ .



Die Hinreaktion entspricht dem Abgeben eines H^+ , also der Deprotonierung des Teilchens auf der linken Seite. Es entstehen ein zurückbleibendes Teilchen mit einer negativeren Ladung und ein freier Wasserstoffkern. Die zweifach positiv geladenen Ionen können schrittweise zwei H^+ abgeben und so einen neutralen Ladungszustand erreichen. Ace und Lac sind neutral, können aber dennoch ein H^+ abgeben und besitzen dann eine negative Ladung.

Innerhalb des Modells gilt das Massenwirkungsgesetz. Das bedeutet, dass die Gesamtkonzentration eines Teilchens und seiner Ione über den Verlauf der Iterationen immer konstant bleibt.

Daraus ergeben sich die Erhaltungsgrößen in diesem Beispiel folgendermaßen

$$E_{\text{Dap}} = C_{\text{Dap}} + C_{\text{Dap}^+} + C_{\text{Dap}^{2+}}$$

$$E_{\text{Dea}} = C_{\text{Dea}} + C_{\text{Dea}^+}$$

$$E_{\text{Tris}} = C_{\text{Tris}} + C_{\text{Tris}^+}$$

$$E_{\text{Imi}} = C_{\text{Imi}} + C_{\text{Imi}^+}$$

$$E_{\text{Bts}} = C_{\text{Bts}} + C_{\text{Bts}^+}$$

$$E_{\text{Pip}} = C_{\text{Pip}} + C_{\text{Pip}^+} + C_{\text{Pip}^{2+}}$$

$$E_{\text{Ace}} = C_{\text{Ace}} + C_{\text{Ace}^+}$$

$$E_{\text{Lac}} = C_{\text{Lac}} + C_{\text{Lac}^+}$$

$$E_{\text{H}} = 2C_{\text{Dap}^{2+}} + C_{\text{Dap}^+} + C_{\text{Dea}^+} + C_{\text{Tris}^+} + C_{\text{Imi}^+} + C_{\text{Bts}^+} + 2C_{\text{Pip}^{2+}} + C_{\text{Pip}^+} + C_{\text{Ace}} + C_{\text{Lac}} + C_{\text{H}^+}.$$

Aus den Reaktionsgleichungen wird außerdem die zugehörige stöchiometrische Matrix

$$S = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{matrix} \text{Dap} \\ \text{Dap}^+ \\ \text{Dap}^{2+} \\ \text{Dea} \\ \text{Dea}^+ \\ \text{Tris} \\ \text{Tris}^+ \\ \text{Imi} \\ \text{Imi}^+ \\ \text{Bts} \\ \text{Bts}^+ \\ \text{Pip} \\ \text{Pip}^+ \\ \text{Pip}^{2+} \\ \text{Ace}^- \\ \text{Ace} \\ \text{Lac}^- \\ \text{Lac} \\ \text{H}^+ \end{matrix}$$

hergeleitet. S hat neunzehn Zeilen für die Reaktanten und zehn Spalten für die Reaktionsgleichungen, in denen jeweils ein Wasserstoffkern und ein negatives Ion durch die Deprotonierung eines der Reaktanten entstehen.

Wir definieren die Reaktionskonstanten $k_i \in \mathbb{R}$ mit $i = 1, \dots, 10$ für jede Reaktionsgleichung und zusätzlich einen Vektor $k \in \mathbb{R}^{10}$ mit den geordneten Reaktionskonstanten als Einträgen.

Dann erhalten wir zehn Flüsse

$$\begin{aligned}
v_{\text{Dap}^+} &= k_1 C_{\text{Dap}^+} - C_{\text{Dap}} C_{\text{H}^+} \\
v_{\text{Dap}^{2+}} &= k_2 C_{\text{Dap}^{2+}} - C_{\text{Dap}^+} C_{\text{H}^+} \\
v_{\text{Dea}^+} &= k_3 C_{\text{Dea}^+} - C_{\text{Dea}} C_{\text{H}^+} \\
v_{\text{Tris}^+} &= k_4 C_{\text{Tris}^+} - C_{\text{Tris}} C_{\text{H}^+} \\
v_{\text{Imi}^+} &= k_5 C_{\text{Imi}^+} - C_{\text{Imi}} C_{\text{H}^+} \\
v_{\text{Bts}^+} &= k_6 C_{\text{Bts}^+} - C_{\text{Bts}} C_{\text{H}^+} \\
v_{\text{Pip}^+} &= k_7 C_{\text{Pip}^+} - C_{\text{Pip}} C_{\text{H}^+} \\
v_{\text{Pip}^{2+}} &= k_8 C_{\text{Pip}^{2+}} - C_{\text{Pip}^+} C_{\text{H}^+} \\
v_{\text{Ace}} &= k_9 C_{\text{Ace}} - C_{\text{Ace}^-} C_{\text{H}^+} \\
v_{\text{Lac}} &= k_{10} C_{\text{Lac}} - C_{\text{Lac}^-} C_{\text{H}^+}
\end{aligned}$$

für die jeweiligen Reaktionsgleichungen. Wir definieren den gesamten Fluss des Systems als einen Vektor $v(C, k) \in \mathbb{R}^{10}$ mit den einzelnen Flüssen der Reaktionsgleichungen als geordnete Einträgen. Mit der Matrix S und dem Vektor $v(C, k)$ erhalten wir, unter Verwendung von (3.3.1), das System gewöhnlicher Differential Gleichungen

$$\dot{C} = Sv(C, k).$$

Dieses simuliert das pH-Puffer Modell und wird von dem Algorithmus in seinen Gleichgewichtszustand überführt. $\dot{C}$ wird dann wie folgt in Python implementiert.

```

1 def fkt_wert(args, x, rel_stoffe):
2     k_h = args[0]
3     k_z = args[1]
4     sm = args[2]
5
6     fw = np.zeros(len(rel_stoffe))
7
8     fw[0] = k_h[0] * x[1] - k_z[0] * x[0] * x[18]
9     fw[1] = k_h[1] * x[2] - k_z[1] * x[1] * x[18]
10    fw[2] = k_h[2] * x[4] - k_z[2] * x[3] * x[18]
11    fw[3] = k_h[3] * x[6] - k_z[3] * x[5] * x[18]
12    fw[4] = k_h[4] * x[8] - k_z[4] * x[7] * x[18]
13    fw[5] = k_h[5] * x[10] - k_z[5] * x[9] * x[18]
14    fw[6] = k_h[6] * x[12] - k_z[6] * x[11] * x[18]
15    fw[7] = k_h[7] * x[13] - k_z[7] * x[12] * x[18]
16    fw[8] = k_h[8] * x[15] - k_z[8] * x[14] * x[18]
17    fw[9] = k_h[9] * x[17] - k_z[9] * x[16] * x[18]
18
19    fw = np.matmul(sm, fw)
20
21    return fw

```


Durch Ableiten nach den einzelnen Reaktanten erhalten wir die dazugehörige Jacobi-Matrix, die folgendermaßen implementiert ist.

```
1 # Berechne Jacobi-Matrix
2 def jacob_i(args, x, rel_stoffe):
3     k_h = args[0]
4     k_z = args[1]
5     sm = args[2]
6
7     jm = np.zeros((len(rel_stoffe), len(x)))
8
9     jm[0, 0] = -k_z[0] * x[18]
10    jm[0, 1] = k_h[0]
11    jm[0, 18] = -k_z[0] * x[0]
12
13    jm[1, 1] = -k_z[1] * x[18]
14    jm[1, 2] = k_h[1]
15    jm[1, 18] = -k_z[1] * x[1]
16
17    jm[2, 3] = -k_z[2] * x[18]
18    jm[2, 4] = k_h[2]
19    jm[2, 18] = -k_z[2] * x[3]
20
21    jm[3, 5] = -k_z[3] * x[18]
22    jm[3, 6] = k_h[3]
23    jm[3, 18] = -k_z[3] * x[5]
24
25    jm[4, 7] = -k_z[4] * x[18]
26    jm[4, 8] = k_h[4]
27    jm[4, 18] = -k_z[4] * x[7]
28
29    jm[5, 9] = -k_z[5] * x[18]
30    jm[5, 10] = k_h[5]
31    jm[5, 18] = -k_z[5] * x[9]
32
33    jm[6, 11] = -k_z[6] * x[18]
34    jm[6, 12] = k_h[6]
35    jm[6, 18] = -k_z[6] * x[11]
36
37    jm[7, 12] = -k_z[7] * x[18]
38    jm[7, 13] = k_h[7]
39    jm[7, 18] = -k_z[7] * x[12]
40
41    jm[8, 14] = -k_z[8] * x[18]
42    jm[8, 15] = k_h[8]
43    jm[8, 18] = -k_z[8] * x[14]
44
45    jm[9, 16] = -k_z[9] * x[18]
46    jm[9, 17] = k_h[9]
47    jm[9, 18] = -k_z[9] * x[16]
48
49    jm = np.matmul(sm, jm)
50
51    return jm
```

Eingabeparameter

Für den anfänglichen Zustand des Systems C_0 nehmen wir an, dass die Konzentration aller neutral geladenen Teilchen gleich zehn ist und die der Ionen gleich 0. Die Konzentration von H^+ ergibt sich aus dem pH-Wert des Puffers.

Bemerkung 3.6. Die Konzentration von H^+ ergibt sich aus dem pH Wert mit $C_{H^+} = 10^{-pH}$. Dafür muss aber die Konzentration in Mol/L gegeben sein. Wir rechnen immer in SI-Basiseinheiten, also Mol/m^3 . Der Unterschied ist ein Faktor von 10^3 . Also wird die Konzentration der H^+ in diesem Beispiel mit $C_{H^+} = 10^{-pH+3}$ berechnet.

Insgesamt ergibt sich

$$C_0 = (10, 0, 0, 10, 0, 10, 0, 10, 0, 10, 0, 10, 0, 0, 0, 10, 0, 10, 10^{-0.5})$$

als anfänglicher Zustand. Die Reaktionskonstanten sind im Vektor

$$k = (2.4 * 10^{-8}, 2.29 * 10^{-6}, 1.31 * 10^{-6}, 8.47 * 10^{-6}, 1.02 * 10^{-4}, 3.28 * 10^{-4}, 1.86 * 10^{-7}, \\ 4.6 * 10^{-3}, 1.75 * 10^{-2}, 1.38 * 10^{-1})$$

gegeben. Zusätzlich sei die erste Zeitschrittweite mit $t_0 = 1$, die maximale Anzahl an Iterationen mit `max_iter` = 50 und die untere Schranke für das Residuum durch `eps` = $1 * 10^{-10}$ gegeben.

Durchführung mit optimaler Pseudo-Zeitschrittweite

Der Algorithmus terminiert nach der fünfzehnten Iteration und gibt die Lösung

$$C_{OPT} = (0.510, 8.132, 1.358, 7.741, 2.259, 9.568, 0.432, 9.963, 0.0375, 9.988, 0.01164, \\ 3.272, 6.727, 5.592 * 10^{-4}, 10, 0, 10, 0, 3.824 * 10^{-7})$$

zurück. Das Residuum an dieser Stelle beträgt ca. $8.4 * 10^{-13}$.

Tabelle 5 zeigt den gesamten Verlauf der Residuen und der Zeitschrittweiten. Wie zu erwarten erhöhen sich die Zeitschrittweiten im Verlauf der Iterationen exponentiell und die Residuen laufen gegen Null. Der Algorithmus hat nach der fünfzehnten Iteration ein Residuum $< \epsilon$ erreicht und terminiert. Die Abbildung 4 gibt einen Überblick über den Verlauf der Konzentrationen der einzelnen Reaktanten.

Jeder neutrale Ladungszustand der Reaktanten ist am Anfang mit einer Konzentration von Zehn vorhanden. Diese Menge wird im Verlauf der Iterationen von den Reaktanten und ihren Ionen eingehalten. Die Erhaltungsgrößen bleiben also tatsächlich erhalten, so dass sich die Werte der einzelnen Ionen wieder zu Zehn aufaddieren. Dieses Ergebnis wird von Tabelle 6 bestätigt.

Der Algorithmus hat das System erfolgreich in seinen Gleichgewichtszustand überführt und dabei die Beschränkung der Erhaltungsgrößen aufrecht erhalten. Das wesentliche Ergebnis ist, dass das entwickelte Verfahren geeignet ist, um Lösungen für komplexe ph-Puffer Modelle zu berechnen. D.h., das chemische System wird erfolgreich in seinen Gleichgewichtszustand überführt.

Iterationen	Residuum	Zeitschrittweite
0	20.671	1.000
1	1.998	8.908
2	0.909	44.336
3	0.225	224.378
4	$4.642 \cdot 10^{-2}$	908.097
5	$5.252 \cdot 10^{-3}$	8,676.497
6	$5.616 \cdot 10^{-4}$	24,408.124
7	$1.448 \cdot 10^{-4}$	$1.149 \cdot 10^5$
8	$3.112 \cdot 10^{-5}$	$3.861 \cdot 10^5$
9	$1.065 \cdot 10^{-5}$	$8.522 \cdot 10^5$
10	$4.121 \cdot 10^{-6}$	$2.07 \cdot 10^6$
11	$1.558 \cdot 10^{-6}$	$5.133 \cdot 10^6$
12	$4.331 \cdot 10^{-7}$	$1.812 \cdot 10^7$
13	$5.936 \cdot 10^{-8}$	$1.295 \cdot 10^8$
14	$1.456 \cdot 10^{-9}$	$5.752 \cdot 10^9$
15	$8.399 \cdot 10^{-13}$	$1.141 \cdot 10^{13}$

Tabelle 5: Entwicklung der Residuen und Zeitschrittweiten aus Beispiel 3

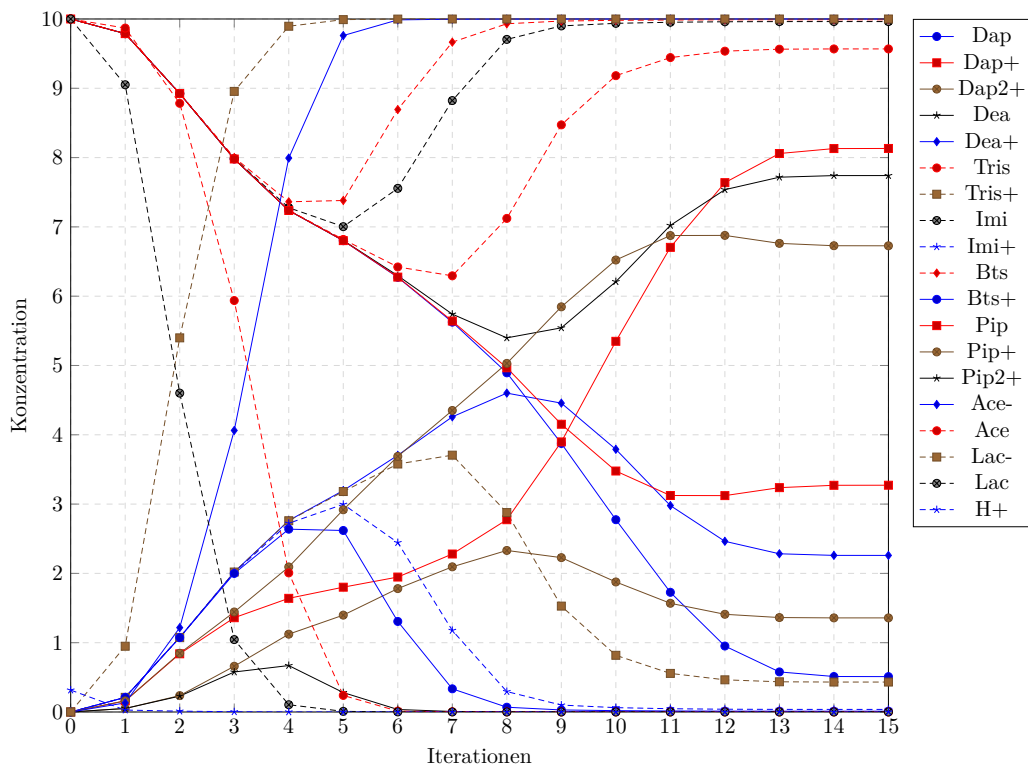


Abbildung 4: Verlauf der Konzentrationen aus Beispiel 3

Itemtionen	Dap	Dap+	Dap2+	Dea	Dea+	Tris	Tris+	Imi	Imi+	Bts	Bts+	Pip	Pip+	Pip2+	Ace-	Ace	Lac-	Lac	H+	Residuum	Zeitschrittweite
0	10.000	0.000	0.000	10.000	0.000	0.000	0.000	10.000	0.000	10.000	0.000	10.000	0.000	0.000	0.000	10.000	0.000	10.000	0.316	20.671	1.000
1	9.789	0.160	$5.075 \cdot 10^{-2}$	9.789	0.211	9.789	0.211	9.789	0.211	9.789	0.211	9.789	0.161	$5.057 \cdot 10^{-2}$	0.131	9.869	0.949	9.951	$2.78 \cdot 10^{-2}$	1.998	8.908
2	8.924	0.840	0.237	8.924	1.076	8.924	1.076	8.925	1.075	8.926	1.074	8.924	0.847	0.229	1.217	8.783	5.399	4.601	$1.238 \cdot 10^{-2}$	0.909	44.336
3	7.980	1.360	0.660	7.980	2.020	7.980	2.020	7.986	2.014	8.001	1.999	7.980	1.443	0.577	4.063	5.937	8.955	1.045	$3.695 \cdot 10^{-3}$	0.225	224.378
4	7.237	1.640	1.123	7.237	2.763	7.240	2.760	7.277	2.723	7.362	2.638	7.237	2.093	0.670	7.993	2.107	9.895	0.105	$7.59 \cdot 10^{-4}$	4.642 $\cdot 10^{-2}$	908.097
5	6.801	1.801	1.398	6.804	3.196	6.819	3.181	7.003	2.997	7.381	2.619	6.802	2.918	0.280	9.761	0.239	9.991	$9.312 \cdot 10^{-3}$	$1.119 \cdot 10^{-4}$	$5.252 \cdot 10^{-3}$	8.676.497
6	6.272	1.947	1.780	6.296	3.704	6.422	3.578	7.556	2.444	8.693	1.307	6.275	3.689	$3.608 \cdot 10^{-2}$	9.987	$1.28 \cdot 10^{-2}$	9.999	$1.393 \cdot 10^{-3}$	$1.768 \cdot 10^{-5}$	$5.616 \cdot 10^{-4}$	24.408.124
7	5.626	2.280	2.095	5.741	4.259	6.294	3.706	8.823	1.177	9.666	0.334	5.641	4.351	$7.656 \cdot 10^{-3}$	9.997	$3.87 \cdot 10^{-3}$	10.000	$4.392 \cdot 10^{-4}$	$6.056 \cdot 10^{-6}$	1.448 $\cdot 10^{-4}$	1.149 $\cdot 10^5$
8	4.894	2.774	2.331	5.399	4.601	7.122	2.878	9.765	0.285	9.931	$6.882 \cdot 10^{-2}$	4.967	5.030	$2.729 \cdot 10^{-3}$	9.999	$1.104 \cdot 10^{-3}$	10.000	$1.399 \cdot 10^{-4}$	$1.93 \cdot 10^{-6}$	$3.112 \cdot 10^{-5}$	$3.861 \cdot 10^5$
9	3.875	3.897	2.227	5.543	4.457	8.472	1.528	9.900	0.100	9.970	$2.962 \cdot 10^{-2}$	4.152	5.846	$1.393 \cdot 10^{-3}$	9.999	$5.489 \cdot 10^{-4}$	10.000	$6.959 \cdot 10^{-5}$	$9.604 \cdot 10^{-7}$	$1.065 \cdot 10^{-5}$	$8.522 \cdot 10^5$
10	2.775	5.349	1.876	6.210	3.790	9.182	0.818	9.937	$6.281 \cdot 10^{-2}$	9.981	$1.949 \cdot 10^{-2}$	3.477	6.522	$9.534 \cdot 10^{-4}$	10.000	$3.652 \cdot 10^{-4}$	10.000	$4.631 \cdot 10^{-5}$	$6.391 \cdot 10^{-7}$	$4.121 \cdot 10^{-6}$	$2.07 \cdot 10^6$
11	1.727	6.705	1.569	7.021	2.979	9.443	0.557	9.953	$4.711 \cdot 10^{-2}$	9.985	$1.468 \cdot 10^{-2}$	3.124	6.875	$7.322 \cdot 10^{-4}$	10.000	$2.753 \cdot 10^{-4}$	10.000	$3.491 \cdot 10^{-5}$	$4.818 \cdot 10^{-7}$	$1.558 \cdot 10^{-6}$	$5.133 \cdot 10^6$
12	0.952	7.638	1.410	7.537	2.463	9.536	0.464	9.960	$4.004 \cdot 10^{-2}$	9.988	$1.248 \cdot 10^{-2}$	3.122	6.877	$6.127 \cdot 10^{-4}$	10.000	$2.342 \cdot 10^{-4}$	10.000	$2.97 \cdot 10^{-5}$	$4.098 \cdot 10^{-7}$	$4.331 \cdot 10^{-7}$	$1.812 \cdot 10^7$
13	0.578	8.059	1.363	7.717	2.283	9.564	0.436	9.962	$3.767 \cdot 10^{-2}$	9.988	$1.175 \cdot 10^{-2}$	3.237	6.762	$5.664 \cdot 10^{-4}$	10.000	$2.304 \cdot 10^{-4}$	10.000	$2.795 \cdot 10^{-5}$	$3.857 \cdot 10^{-7}$	$5.936 \cdot 10^{-8}$	$1.295 \cdot 10^8$
14	0.512	8.130	1.358	7.740	2.260	9.568	0.432	9.963	$3.735 \cdot 10^{-2}$	9.988	$1.165 \cdot 10^{-2}$	3.271	6.728	$5.594 \cdot 10^{-4}$	10.000	$2.185 \cdot 10^{-4}$	10.000	$2.771 \cdot 10^{-5}$	$3.824 \cdot 10^{-7}$	$1.456 \cdot 10^{-9}$	$5.752 \cdot 10^9$
15	0.510	8.132	1.358	7.741	2.259	9.568	0.432	9.963	$3.735 \cdot 10^{-2}$	9.988	$1.164 \cdot 10^{-2}$	3.272	6.727	$5.592 \cdot 10^{-4}$	10.000	$2.185 \cdot 10^{-4}$	10.000	$2.771 \cdot 10^{-5}$	$3.824 \cdot 10^{-7}$	$8.399 \cdot 10^{-13}$	$1.141 \cdot 10^{13}$

Tabelle 6: Entwicklung der Konzentrationen aus Beispiel 3

4. Robustheit des PTC Algorithmus

4.1. Robustheit

Die Robustheit eines Verfahrens beschreibt dessen Fähigkeit, störenden Einflüssen von außen zu widerstehen. In der Entwicklung von numerischen Verfahren ist es ein wesentlicher Faktor in der Beurteilung der Qualität von Algorithmen.

Einer der großen Vorteile von Pseudo Transient Continuation ist es, dass die Entfernung des Startpunktes zum Gleichgewichtspunkt keine, oder nur eine sehr kleine, Auswirkung auf die Konvergenz des Verfahrens hat. Um diese Eigenschaft für das von mir implementierte Verfahren zu testen, wird der in 3.2 vorgestellte Algorithmus mit einem Newton-Verfahren verglichen. Zu diesem Zweck wird der NLEQ_RES Algorithmus aus Kapitel 3 in [Deu11] verwendet.

4.2. Vergleich mit einem Newton Verfahren

Um die Annahme zu testen, dass Pseudo Transient Continuation im Vergleich zum Newton-Verfahren global konvergiert, wird eine von [Lew] zur Verfügung gestellte Implementierung von Newton verwendet. Das NLEQ_RES Verfahren ist ein residuum-basiertes, adaptives Trust-Region Newton-Verfahren. Die Anwendung erfolgt auf Problemstellungen für das Steric Mass Action (SMA) Modell aus Abschnitt 3.4.2.

Beide Verfahren werden zu diesem Zweck auf Konvergenz in Abhängigkeit von der Entfernung eines übergebenen Startzustandes zum Gleichgewichtspunkt $\mathbf{x}_{\text{ref}}$ getestet. Dafür wird eine untere Schranke `normmin` definiert und angenommen, dass ein Verfahren genau dann erfolgreich konvergiert ist, wenn die approximierte Lösung $\mathbf{x}_n$ ein Residuum kleiner als diese Schranke besitzt.

Um einen aussagekräftigen Vergleich zwischen den beiden Verfahren ziehen zu können, müssen diese für eine große Anzahl von Punkten mit selbem Abstand zur Gleichgewichtslösung getestet werden. Hierfür wird eine sphärische Schale, mit innerem Radius `r_min` um $\mathbf{x}_{\text{ref}}$ definiert. Der äußere Radius der Schale `r_max` wird größer, aber ausreichend nah an `r_min` gewählt, damit angenommen werden kann, dass alle Punkte innerhalb der Schale denselben Abstand zu $\mathbf{x}_{\text{ref}}$ besitzen. Dann wird eine große Anzahl beliebiger, gleich-verteilter Punkte erzeugt, die alle innerhalb der Schale liegen. Im Verlauf des Tests werden die Radien schrittweise erhöht und so neue Schalen mit neuen Punkten erzeugt, die einen größeren Abstand zu $\mathbf{x}_{\text{ref}}$ besitzen. In jedem Schritt wird für die erzeugte Schale festgehalten, wie viele der Punkte bei welchem Verfahren zu Konvergenz geführt haben. Die Ergebnisse des Tests werden am Ende miteinander verglichen um zu zeigen, dass PTC im Gegensatz zu Newton tatsächlich invariant gegenüber dem Abstand des Startzustandes zu $\mathbf{x}_{\text{ref}}$ ist.

Implementierung in Python

Die `points` Methode, wird mir von [Lew] zur Verfügung gestellt und erzeugt innerhalb einer sphärischen Schale mit festgelegter Radien `r_min` und `r_max`, gleich-verteilte Punkte. Der Wert `nSamples` legt die Anzahl der erzeugten Punkte fest. Es entsteht eine Umgebung mit gleich-verteilten Punkten, welche alle einen Abstand zwischen `r_min` und `r_max` zur Gleichgewichtslösung besitzen.

```
1 # Erzeuge Kugel mit gleichverteilten Punkten
2 def points(nDim, nSamples, r_min, r_max):
3     # Richtung ziehen
4     dirs = np.random.normal(size=(nDim, nSamples))
5     norms = np.linalg.norm(dirs, axis=0)
6     dirs = dirs / norms
7
8     # Radius ziehen
9     rad = np.power(np.random.uniform(
10         low=r_min ** nDim, high=r_max ** nDim,
11         size=(nSamples,)), 1.0 / nDim)
12
13     return dirs * rad
```

Als Anpassung an die spezifischen Parameter des Beispiels wird die erzeugte Schale mit einer Ellipse `sphToEllips` transformiert. Durch diese Transformation besitzen die erzeugten Punkte eine ähnliche Form, was die Ergebnisse des Tests verbessert. Dieser Schritt ist nötig, da der erste Eintrag von `x_ref` deutlich größer als die anderen ist und

```
1 # Erzeuge Punkte und transformiere in eine Ellipse
2 sphToEllips = np.diag([1000, 10, 10, 10])
3 pts = points(nDim, nSamples, r_min, r_max)
4 pts = np.dot(sphToEllips, pts)
```

Der Test beginnt, indem die anfangs gewählten Radien um die Schrittweite `stepsize` erhöht werden und innerhalb der neuen Radien `n_Samples` Punkte erzeugt werden. Für jeden der erzeugten Punkte wird überprüft, ob er den im Beispiel geltenden physikalischen Limitierungen unterliegt. Da in diesem Beispiel Konzentrationen betrachtet werden, sind alle Punkte die einen negativen Eintrag besitzen automatisch nicht sinnvoll und müssen auch nicht überprüft werden. Falls ein Punkt abgelehnt wird, wird mit dem Zähler `nCounter` festgehalten, um wie viel die Anzahl der in dieser Iteration insgesamt Überprüften Punkte abgenommen hat.

```
1 for i in range(0, nSamples):
2     x = []
3     skip = False
4
5     # Für jeden Punkt gehe von x_ref aus und erhalte neuen Punkt
6     for j in range(0, nDim):
7         x_j = x_ref[j] + pts[j, i]
8         if x_j < 0:
9             skip = True
10            x.append(x_j)
11
12    # Entferne negative Punkte
13    if skip:
14        nCounter = nCounter - 1
```

Wurde ein Punkt überprüft und besitzt keinen negativen Eintrag, werden beide Verfahren für ihn auf Konvergenz getestet. Dabei ist `Bsp2Rob` eine leichte Modifikation des schon gezeigten Algorithmus, die für einen übergebenen Startpunkt `x` nur das Residuum an der approximierten Lösung zurückgibt.

Bemerkung 4.1. Wie schon in Bemerkung 3.4 festgehalten, eignet sich in der Praxis SER besser für Probleme mit diesem Modell. Aus diesem Grund operiert `Bsp2Rob` mit Hilfe der Methode `t_ser` aus Kapitel 3.

`SmaTestRob` hingegen benutzt den von [Lew] implementierten `NLEQ_RES` Algorithmus. Wenn eines der zurück gegebenen Residuen einen Wert kleiner `normmin` besitzen, wird die erfolgreiche Konvergenz mit Hilfe der korrespondierenden Zähler `counterPTC` und `counterNEW` festgehalten. Zusätzlich wird überprüft, ob die beiden Verfahren einen legitimen Wert für die Norm oder `NaN` übergeben haben. Geben beide Verfahren für einen Punkt `NaN` zurück, wird auch dieser als nicht geeignet eingestuft und `nCounter` verringert.

```

1      # Ruft Bsp2-Verfahren auf und gibt Residual Norm zurück
2      norm1 = Bsp2_Rob.main(x)
3      if norm1 < normmin:
4          counterPTC = counterPTC + 1
5
6      # Ruft Vergleichsverfahren auf und gibt Residual Norm zurück
7      try:
8          norm2 = SmaTest_Rob.smaTestRob(x)
9      except ValueError:
10         norm2 = float('nan')
11
12     if norm2 < normmin:
13         counterNEW = counterNEW + 1
14
15     if np.isnan(norm1) and np.isnan(norm2):
16         nCounter = nCounter - 1

```

Wenn alle Punkte aus der derzeitigen Ellipse behandelt sind, ergeben sich drei Werte, die das Ergebnis dieser Iteration beinhalten. Die beiden Zähler `counterPTC` und `counterNEW` drücken aus, für wie viele der Punkte das zugehörige Verfahren tatsächlich konvergiert ist. Der Zähler `nCounter` ist die Anzahl der erzeugten Punkte für welche der Test tatsächlich durchgeführt wurde. Fällt der Wert für `nCounter` im Laufe der Iterationen unter die Schranke `nMin`, werden keine weiteren Ellipsen erzeugt. Ein Wert für `nMin` wird festgelegt, um zu garantieren, dass genügend legale Punkte übrig sind um eine brauchbare Aussage über das Verhalten der Verfahren zu ermöglichen. Waren im letzten Schritt noch genügend Punkte verfügbar, startet der Prozess mit einer neu erzeugten Menge von Punkten innerhalb der nächsten Ellipse mit erhöhten Radien von neuem.

```

1      # Berechne Anzahl der konvergierten legalen Punkte
2      wert1 = counterPTC / nCounter
3      wert2 = counterNEW / nCounter
4
5      # Erzeuge df mit Prozent der konvergierten Punkte
6      # ... für Ausgabe als Plot
7      df_g.loc[step] = [wert1, wert2]
8      # ... und für Ausgabe als Tabelle
9      df_t.loc[step] = [wert1, wert2, nCounter, r_min]
10     df_t.index.name = 'Iterationen'
11
12     return df_g, df_t

```

Der Prozess wird wiederholt bis entweder die vorgegebene maximale Anzahl an Schritten **maxDistance** erreicht wurde oder nicht mehr genügend legale Punkte verfügbar sind. In diesem Fall werden die Ergebnisse der bisherigen Iterationen ausgewertet. Die Werte **wert1** und **wert2** beschreiben prozentual den Anteil der Punkte für welche die Verfahren konvergiert sind. Dabei wird die Anzahl der unzulässigen Punkte durch Verwendung von **nCounter** berücksichtigt.

Eingabeparameter

Der Test wird mit den folgenden Parametern durchgeführt:

```

nSamples = 100000
nMin = 1000
r_min = 0
r_max = 0.1
stepsize = 0.1
normmin = 0.02
maxDistance = 301

```

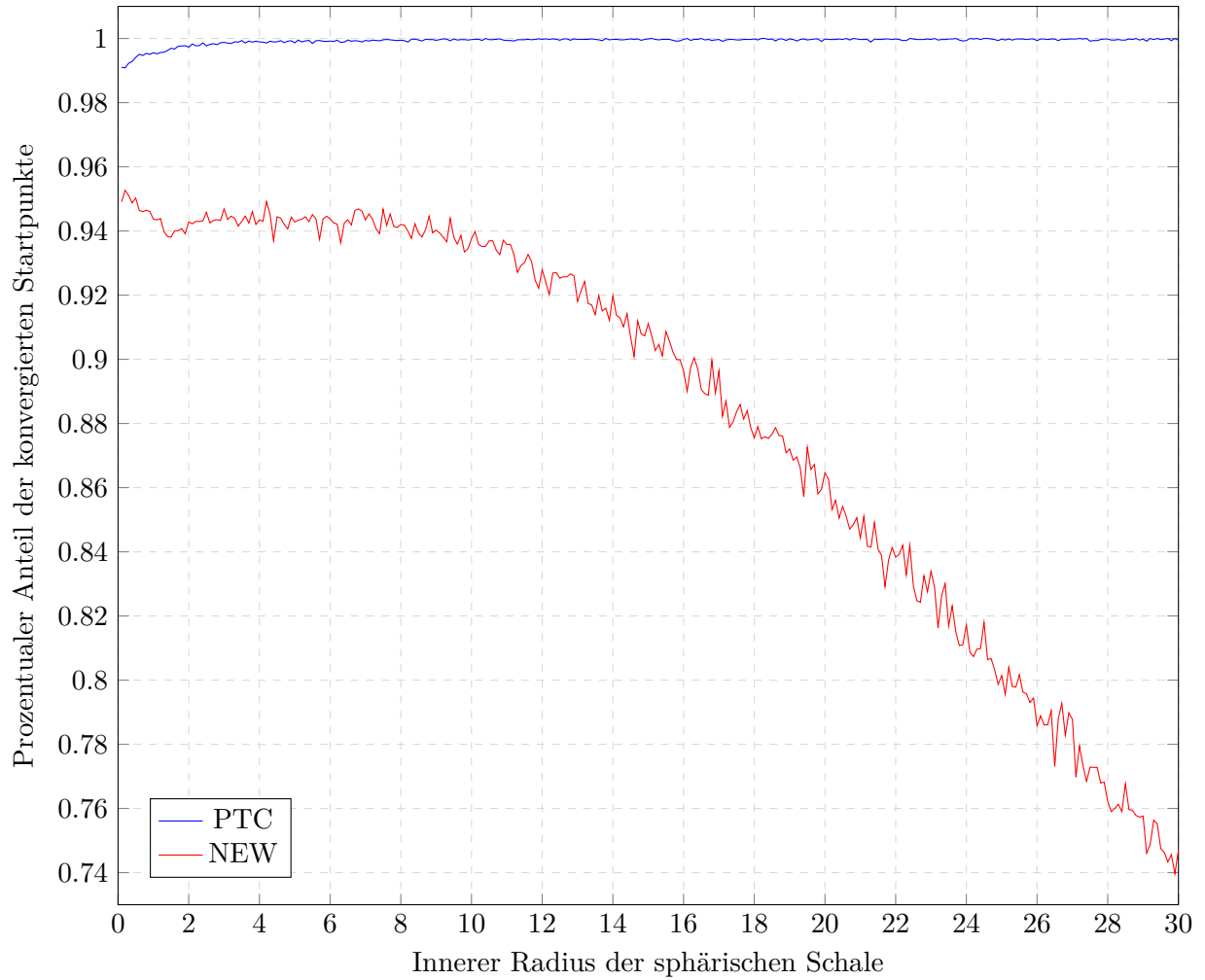



Abbildung 5: Vergleich der beiden Verfahren

Ergebnisse

Abbildung 5 gibt einen Überblick über die Performance der beiden Verfahren im Verlauf von 301 Iterationen. In jedem Iterationsschritt werden mit `stepsize` die Radien um 0.1 erhöht. Auf der x-Achse steht der innere Radius $r_{\min}$ der erzeugten Ellipse (der Abstand zu $\mathbf{x}_{\text{ref}}$), und auf der y-Achse der prozentuale Anteil von Punkten (mit diesem Abstand), für welche die Verfahren konvergiert sind. Ein Verfahren gilt als konvergiert, wenn es an seiner approximierten Lösung ein Residuum < 0.02 aufweist. PTC steht für Pseudo Transient Continuation und NEW für NLEQ_RES.

Am Verlauf der Kurve von PTC ist zu erkennen, dass das Verfahren in den meisten Iterationen eine fast perfekte Erfolgsquote besitzt. Im Verlauf sind nur kleine Schwankungen zu erkennen und es ist keine Entwicklung nach unten erkenntlich.

Bemerkung 4.2. Der Leichte Anstieg in der Erfolgsquote in den ersten 40 Iterationen hat seinen Ursprung in der verwendeten Methode zur Zeitweiten Bestimmung. Für manche Punkte nahe $\mathbf{x}_{\text{ref}}$ wird in den ersten Iterationen von PTC die Zeitschrittweite nur sehr langsam erhöht oder sogar leicht verringert, so dass das Verfahren direkt zu Anfang abbricht. Um diesen

Effekt zu minimieren habe ich PTC erlaubt in den ersten Iterationsschritten ohne Reduktion des Residuums weiter zu verfahren, eine heuristisches Strategie aus Abschnitt 2.3.

Die Kurve zu NLEQ_RES zeigt im Gegensatz dazu, dass dieses Verfahren in einer Umgebung um $\mathbf{x}_{\text{ref}}$ seine beste Performance erzielt und sich anschließend immer weiter verschlechtert. Die Kurve weist deutliche Schwankungen von einer Ellipse zur nächsten auf, die sich mit steigendem Abstand zu $\mathbf{x}_{\text{ref}}$ vergrößern.

Insgesamt bestätigen die Ergebnisse die Behauptung, dass bei Verwendung von PTC die Entfernung des Startpunktes zu $\mathbf{x}_{\text{ref}}$ keine wichtige Rolle spielt. Das Newton-Verfahren verschlechtert im Gegensatz seine Performance mit steigendem Abstand deutlich. Diese Eigenschaft ist einer der größten Vorteile von PTC und einer der Hauptaussagen die in dieser Arbeit gezeigt werden.

5. Fazit und Ausblick

Pseudo Transient Continuation ist ein lineares implizites Euler-Verfahren, das nichtlineare Probleme $F(x) = 0$ löst, in dem der Gleichgewichtspunkt des daraus resultierenden Systems gewöhnlicher Differentialgleichungen

$$\dot{x} = \frac{dx}{dt} = F(x), x(0) = x_0$$

berechnet wird. Die dem Verfahren zugrunde liegende Idee ist, eine Pseudo-Zeit einzuführen und das System bis zu seinem Gleichgewichtspunkt $\dot{x} = F(x) = 0$ zu integrieren. Die Pseudo-Zeit erlaubt große, sich im Laufe der Iterationen erhöhende Zeitschrittweiten, mit deren Hilfe der Gleichgewichtspunkt schnell erreicht wird.

In dieser Arbeit wurde immer vorausgesetzt, dass eine exakt bestimmte Jacobi-Matrix zur Verfügung steht und das in jeder Iteration zu lösende lineare Gleichungssystem sich mit Hilfe von exakten Eliminations-Methoden lösen lässt. Das Verfahren kann mit geeigneter Anpassung allerdings auch mit approximierter Jacobi-Matrix durchgeführt werden. Auch in Fällen, in denen die Lösung des linearen Systems sich nicht mit exakten Methoden bestimmen lässt, kann das Verfahren mit iterativen Lösungsmethoden angepasst werden. Beide Fälle werden in Kapitel 6 aus [Deu11] genauer beschrieben.

Die Wahl des eingeführten Pseudo-Zeitschrittes wurde in dieser Arbeit entweder mit der theoretisch optimalen Strategie berechnet, oder alternativ mit Switched Evolution Relaxation. Neben diesen beiden in der Arbeit behandelten Strategien existieren weitere heuristische Strategien, von denen z.B. manche in [KK98] vorgestellt werden.

Der PTC Algorithmus wurde vorgestellt und in Python implementiert. Wesentliche Bestandteile des Quellcodes wurden detailliert erklärt und gezeigt, wie chemische Systeme mathematisch modelliert werden. Zu diesem Zweck wurden mehrere Hilfsmittel aus der Chemie eingeführt, mit denen alle nötigen Informationen aus den Reaktionsgleichungen eines Systems erhalten werden können. Eine Einführung in die mathematische Modellierung chemischer Modelle und den benötigten Hilfsmitteln bietet z.B. [Hil09]. Der Pseudo Transient Continuation Algorithmus bestimmt dann den Gleichgewichtszustand des erzeugten Modells.

Anschließend wurde der PTC Algorithmus am Beispiel verschiedener chemischer Systeme getestet. Für jedes Beispiel wurde gezeigt, wie das Modell konstruiert wurde und wie das Modell anschließend mit dem Algorithmus in seinen Gleichgewichtszustand überführt wurde. Dabei wurden verschiedene heuristische Strategien für die Bestimmung der Pseudo-Zeitschrittweiten verwendet und verglichen. Insgesamt wurde bestätigt, dass das Verfahren schneller gegen die Gleichgewichtslösung konvergiert, wenn die Zeitschrittweite schneller erhöht wird.

Im letzten Teil der Arbeit wurde getestet wie robust der PTC Algorithmus gegenüber der Wahl des Startzustandes ist, speziell im Vergleich mit einem Newton-Verfahren. Hierzu wurden beide Verfahren für eine große Anzahl gleich-verteilter Punkte mit steigenden Abständen zum Gleichgewichtszustand getestet. Am Ende wurde ausgewertet, für welchen Prozentsatz der Punkte mit gleichem Abstand die beiden Verfahren jeweils konvergiert sind. Aufgrund seiner lokalen Konvergenz verschlechterte sich das Newton-Verfahren mit steigendem Abstand deutlich. Der implementierte PTC Algorithmus hingegen behielt durchgängig eine sehr gute Konvergenz-Rate.

Literatur

- [BC92] Clayton A. Brooks and Steven M. Cramer. Steric mass-action ion exchange: Displacement profiles and induced salt gradients. *AIChE Journal*, 38(12):1969–1978, 1992.
- [Deu11] Peter Deuffhard. *Newton Methods for Nonlinear Problems: Affine Invariance and Adaptive Algorithms*. Springer Publishing Company, Incorporated, 2011.
- [Hil09] Uwe Hillebrand, editor. *Stöchiometrie : Eine Einführung in die Grundlagen mit Beispielen und Übungsaufgaben*. Springer-Lehrbuch SpringerLink. Springer Berlin Heidelberg, Berlin, Heidelberg, 2 edition, 2009.
- [Joh19] Robert Johansson. *Numerical Python : Scientific Computing and Data Science Applications with Numpy, SciPy and Matplotlib*. SpringerLink. Apress, Berkeley, CA, 2nd ed. edition, 2019.
- [KK98] C. T. Kelley and David E. Keyes. Convergence analysis of pseudo-transient continuation. *SIAM Journal on Numerical Analysis*, 35(2):508–523, 1998.
- [Lew] Samuel Leweke. Persönliche Kommunikation.
- [Sch19] Christoph Schäfer. *Schnellstart Python: Ein Einstieg ins Programmieren für MINT-Studierende*. essentials Springer eBooks. Springer Spektrum, Wiesbaden, 2019.

A. Notationsverzeichnis

Kapitel 2	
$F(x)$	Nichtlineares Problem
$F'(x)$	Jacobi-Matrix eines nichtlinearen Problems
$G(y), G'(y)$	Transformiertes nichtlineares Problem und seine Jacobi-Matrix
$\dot{x}, \dot{y}$	Ableitung
$J, J(x)$	Jordan-Normalform einer Matrix
$ \cdot , \langle \cdot, \cdot \rangle$	Euklidische Norm und Euklidisches Skalarprodukt
$\ \cdot\ , (\cdot, \cdot)$	Kanonische Norm und Kanonisches Skalarprodukt
x_0, y_0	Startwerte einer Iteration
$\Delta x, \Delta y$	Änderungsrate einer Iteration
$x(\tau)$	Abbildung die den Weg beginnend von einem Startpunkt aus beschreibt
$F(x(\tau))$	Residuum
D	Definitionsbereich (einer Funktion)
I	Identitätsmatrix
e	Vektor mit nur Eins Einträgen
A	Alternative Schreibweise für die Jacobi-Matrix
B	Beliebige invertierbare Matrix
T	Transformationsmatrix
t	Zeitschrittweite
τ	Pseudo-Zeitschrittweite
τ^*	Eine bestimmte Pseudo-Zeitschrittweite größer Null
γ	Lokale Zeitschrittweite
λ	Eigenwert
$\lambda_{\max}$	Größter Eigenwert
κ	Wert zwischen Null und Eins
k	Iterations Index bei Mehrschrittverfahren
L	Lipschitz-Konstante
u	Vektor aus dem $\mathbb{R}^n$
$\tilde{u}$	Skalierter Vektor u
i	Index aus $\mathbb{N}$
ϵ	Ein Wert größer Null
$P, P^\perp$	Orthogonale Projektion
S_P	Unterraum mit allen Vektoren die mit $P^\perp$ auf Null abbilden
$\alpha, \alpha(\tau), \dot{\alpha}(\tau)$	Kurze Schreibweise für einen Term
$\mu, \mu(A)$	Einseitige Lipschitz-Bedingung in der kanonischen Norm
$\nu, \nu(A), \tilde{\nu}, \tilde{\nu}(\tau)$	Einseitige Lipschitz-Bedingung in der euklidischen Norm
L	Lipschitz-Bedingung 2. Ordnung
$[\nu], [L]$	Rechnerische Näherungen für die Lipschitz-Konstanten
F_0	Kurze Schreibweise für $\ F'(x_0)\ $
$\tau_{\text{opt}}, [\tau_{\text{opt}}]$	Optimale Pseudo-Zeitschrittweite und ihre rechnerische Näherung
$\bar{\tau}_{\text{opt}}$	Obere Schranke für die optimalen Pseudo-Zeitschrittweite
τ_{ser}	Pseudo-Zeitschrittweite, die mit SER bestimmt wird

Kapitel 3

$F(x)$	Nichtlineares Problem
$F'(x)$	Jacobi-Matrix eines nichtlinearen Problems
x	Zustand des Systems
x_0	Anfänglicher Zustand des Systems
τ	Pseudo-Zeitschrittweite
τ_0	Anfängliche Pseudo-Zeitschrittweite
i	Index für die Anzahl der Iterationen, oder Anzahl Gleichungen
ϵ	Untere Schranke für das Residuum
Δx	Änderungsrate des Zustands des Systems
I	Identitätsmatrix
OPT	Optimale Zeitschrittweite
SER	Switched Evolution Relaxation
max_iter	Maximale Anzahl an Iterationen die der Algorithmus durchführt
delta_x	Methode zur Berechnung der Änderung des Systems in der aktuellen Iteration
dx	Berechnete Änderung des Zustands
solve()	Methode aus der Python Bibliothek zum lösen linearer Gleichungssysteme
k	Vektor mit benutzerdefinierten Parametern
t_0	Anfänglich gewählte Zeitschrittweite
t_n	Zeitschrittweite der aktuellen Iteration
x, x_0	Vektor mit dem Zustand des Systems
x_n	Vektor mit dem aktuellen Stand des Systems
x_n1	Vektor mit dem nächsten berechneten Zustand des Systems
t_opt	Methode zum Berechnen der nächsten Zeitschrittweite mit OPT
t_ser	Methode zum Berechnen der nächsten Zeitschrittweite mit SER
stoffe	Geordnete Liste mit den Namen der Reaktanten
opt	Boolean Variable, die Art der Bestimmung der Zeitschrittweite bestimmt
$S, S_{i,j} \in \mathbb{R}^{m \times n}$	Stöchiometrische Matrix mit m Reaktanten und n Reaktionsgleichungen
$C, C(t)$	Konzentrationen
$\dot{C}, \dot{C}(t)$	System gewöhnlicher Differentialgleichungen
$v, v(C), c(C, k)$	Vektor mit den Flüssen der einzelnen Reaktionen als Einträgen
A, B, C	Reaktanten
C_a, C_b, C_c	Konzentrationen der Reaktanten
k	Vektor mit Reaktionskonstanten und seine Einträge
$k_{a,i}, k_{d,i}, k_i$	Reaktionskonstanten für Hin- und Rückrichtung
C_0	Startparameter für den Zustand des Systems
t_0	Startparameter für die Zeitschrittweite
max_iter	Startparameter für die maximale Anzahl an Iterationen
C_{OPT}	Berechnete Lösung mit OPT
C_{SER}	Berechnete Lösung mit SER
SMA	Steric Mass-Action Modell
Λ	Kapazität der Oberfläche im SMA Modell
ν	Charakteristische Ladung eines Proteins
σ	Sterischer Faktor eines Proteins

Kapitel 3 - Fortsetzung	
P	Protein (gelöst in der Flüssigkeit)
FS	Freie Bindungsstelle auf der Oberfläche, besetzt mit einem Salz-Ion
S	Salz-Ion (gelöst in der Flüssigkeit)
BP	Ein an die Oberfläche gebundenes Protein
C	Konzentration in der Flüssigkeit
Q	Konzentration auf der Oberfläche
$()^s$	Konzentration von Salz-Ionen
$()^p$	Konzentration von Proteinen
$\bar{Q}$	Verfügbare Bindungsstellen auf der Oberfläche
$\hat{Q}$	Verdeckte Bindungsstellen auf der Oberfläche
H ⁺	Einfach positiv geladener Wasserstoffkern
Dap, Dea, Tris, Imi, Bts, Pip, Ace und Lac	Reaktanten
Dap ⁺ , Dap ²⁺ , Dea ⁺ , Tris ⁺ , Imi ⁺ , Bts ⁺ , Pip ⁺ , Pip ²⁺ , Ace ⁻ , Lac ⁻	Ione der Reaktanten
$E_{\text{Dap}}, E_{\text{Dea}}, E_{\text{Imi}}, E_{\text{Bts}}, E_{\text{Pip}}, E_{\text{Ace}}, E_{\text{Lac}}, E_{\text{H}}$	Erhaltungsgrößen
$v_{\text{Dap}^+}, v_{\text{Dap}^{2+}}, v_{\text{Dea}^+}, v_{\text{Tris}^+}, v_{\text{Imi}^+}, v_{\text{Bts}^+}, v_{\text{Pip}^+}, v_{\text{Pip}^{2+}}, v_{\text{Ace}}, v_{\text{Lac}^+}$	Flüsse der einzelnen Reaktionsgleichungen
Kapitel 4	
NLEQ_RES	Residuum basierender Newton Algorithmus
SMA	Steric Mass Action
PTC	Pseudo Transient Continuation
NEW	Newton
normmin	Untere Schranke für das Residuum
x_n	Derzeitiger Zustand des Systems
r_min, r_max	Innerer und Äußerer Radius einer sphärischen Schale
n_Samples	Anzahl der erzeugten Punkte innerhalb einer Schale
sphToEllips	Methode für die Umwandlung einer sphärischen Schale in Ellipsenform
x_ref	Gleichgewichtszustand des Systems
stepsize	Wert um den die Radien in jedem Schritt erhöht werden
n_Counter	Zähler, der sich merkt wie viele zulässige Punkte getestet wurden
Bsp2Rob	Berechnet eine Lösung mit PTC
SmaTestRob	Berechnet eine Lösung mit Newton
counterPTC	Exakte Anzahl von Punkten für die PTC konvergiert ist
counterNEW	Exakte Anzahl von Punkten für die NEW konvergiert ist
NaN	Rückgabewert, falls keine gültige Zahl
nMin	Minimal nötige Anzahl von legalen Punkten um noch einen Schritt durchzuführen
maxDistance	Maximale Anzahl an sphärischen Schalen die erzeugt werden sollen
wert1, wert2	Geben prozentual an, wie viele der Punkte konvergiert sind
nDim	Legt Anzahl der Reaktanten im System fest

B. Quellcode

B.1. Beispiel 1

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import pandas as pd
4 from numpy import linalg as la
5
6
7 # Berechne Funktionswerte
8 def fkt_wert(k, x):
9     return np.array(
10         [-k[0] * x[0] * x[1] + k[1] * x[2],
11          -k[0] * x[0] * x[1] + k[1] * x[2],
12          k[0] * x[0] * x[1] - k[1] * x[2]])
13
14
15 # Berechne Jacobi-Matrix
16 def jacobi(k, x):
17     return np.array(
18         [[-k[0] * x[1], -k[0] * x[0], k[1]],
19          [-k[0] * x[1], -k[0] * x[0], k[1]],
20          [k[0] * x[1], k[0] * x[0], -k[1]]])
21
22
23
24 # Berechne die Änderungsrate Delta x
25 def delta_x(k, t, x):
26     i = np.eye(len(x))
27     dx = la.solve(i - t * jacobi(k, x), fkt_wert(k, x))
28     return dx
29
30
31 # Berechne optimalen Zeitschritt
32 def t_opt(k, t_n, x_n, x_n1, dx):
33     z = abs(np.dot(dx, fkt_wert(k, x_n) - dx))
34     n = 2 * la.norm(dx) * la.norm(fkt_wert(k, x_n1) - dx)
35     return (z / n) * t_n
36
37
38 # Berechne SER Zeitschritt
39 def t_ser(k, t_n, x_n, x_n1):
40     z = t_n * la.norm(fkt_wert(k, x_n))
41     n = la.norm(fkt_wert(k, x_n1))
42     return z / n
43
44
45 # Zeige df
46 def plot_df(df):
47     print(df)
48     df.plot()
49     plt.show()
50     return
51
52
53 # Schreibe df als csv in das LaTeX-Verzeichnis
54 def write_df(df, path, file):
```



```

55     with open(path + file, 'w') as tf:
56         tf.write(df.to_csv())
57     return
58
59
60 # Berechne Beispiel
61 def calc_bsp(k, x0, t_n, stoffe, max_iter, opt, eps):
62     x_n = x0
63     x_list = [x0]
64     t_list = [t_n]
65     res_list = [la.norm(fkt_wert(k, x0))]
66     i=0
67     while i in range(0, max_iter) and la.norm(fkt_wert(k, x_n)) > eps:
68         dx = delta_x(k, t_n, x_n)
69         x_n1 = x_n + t_n * dx
70         if la.norm(fkt_wert(k, x_n1)) >= la.norm(fkt_wert(k, x_n)):
71             print('Abbruch: Keine Residual-Reduktion')
72             break
73         if opt:
74             t_n1 = t_opt(k, t_n, x_n, x_n1, dx)
75         else:
76             t_n1 = t_ser(k, t_n, x_n, x_n1)
77         t_list = np.vstack((t_list, t_n1))
78         res_list = np.vstack((res_list, la.norm(fkt_wert(k, x_n1))))
79         x_list = np.vstack((x_list, x_n1))
80         t_n = t_n1
81         x_n = x_n1
82         i = i + 1
83     df = pd.DataFrame(data=x_list, columns=stoffe)
84     df['Residuum'] = res_list
85     df['Zeitschrittweite'] = t_list
86     df.index.name = 'Iterationen'
87
88     return df
89
90
91 def main():
92     # Reaktionskonstanten
93     k = np.array([0.2, 0.3])
94
95     # Anfängliche Konzentrationen
96     x0 = np.array([0.25, 1.25, 0.75])
97
98     # Anfänglich gewählter Zeitschritt
99     t_n = 1
100
101     eps = 1e-10
102
103     max_iter = 50
104
105     stoffe = ['$C_a$', '$C_b$', '$C_c$']
106     stoffe2 = ['$C_A$', '$C_B$', '$C_C$']
107
108     for opt in [True, False]:
109         path = "../ba_nv_latex/"
110         if opt:
111             file = 'Data_Bsp1OPT.csv'
112             print("Beispiel: 1 OPT")
113             df1 = calc_bsp(k, x0, t_n, stoffe, max_iter, opt, eps)

```

```

114         dfo = df1.loc[:, 'Residuum']
115         write_df(df1, path, file)
116     else:
117         file = 'Data_Bsp1SER.csv'
118         print("\nBeispiel: 1 SER")
119         df2 = calc_bsp(k, x0, t_n, stoffe2, max_iter, opt, eps)
120         dfs = df2.loc[:, 'Residuum']
121         write_df(df2, path, file)
122
123     file = 'Data_Bsp1.csv'
124     dfm = pd.merge(df1, df2, on='Iterationen', how='outer')
125     write_df(dfm, path, file)
126     plot_df(dfm)
127
128     file = 'Data_Bsp1RES.csv'
129     dfr = pd.merge(dfo, dfs, on='Iterationen', how='outer')
130     dfr.columns = ['OPT', 'SER']
131     plot_df(dfr)
132     write_df(dfr, path, file)
133
134     return
135
136
137 if __name__ == "__main__":
138     main()

```

B.2. Beispiel 2

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import pandas as pd
4 from numpy import linalg as la
5
6
7 # Berechne Jacobi-Matrix
8 def jacobi(args, x):
9     yCp = args[0]
10    kA = args[1]
11    kD = args[2]
12    nu = args[3]
13    sigma = args[4]
14    # Lambda = args[5]
15
16    q0bar = x[0] - np.sum(sigma * x)
17    c0powNu = np.power(yCp[0], nu)
18    # q0barPowNu = np.power(q0bar, nu)
19    q0barPowNuM1 = np.power(q0bar, nu - 1.0)
20    extSigma = -sigma
21    extSigma[0] = 1.0
22
23    jm = np.zeros((len(x), len(x)))
24    jm[0, 0] = -1.0
25    jm[0, 1:] = -nu[1:]
26
27    for i in range(1, len(x)):
28        jm[i, :] = +kA[i] * yCp[i] * nu[i] * q0barPowNuM1[i] * extSigma
29        jm[i, i] -= kD[i] * c0powNu[i]
30
31    return jm
32
33
34 # Berechne Funktionswerte
35 def fkt_wert(args, x):
36     yCp = args[0]
37     kA = args[1]
38     kD = args[2]
39     nu = args[3]
40     sigma = args[4]
41     Lambda = args[5]
42     q0bar = x[0] - np.sum(sigma * x)
43     c0powNu = np.power(yCp[0], nu)
44     q0barPowNu = np.power(q0bar, nu)
45
46     fw = np.zeros(len(x))
47     fw[0] = -x[0] + Lambda - np.sum(nu * x)
48     fw[1:] = -kD[1:] * x[1:] * c0powNu[1:] + kA[1:] * yCp[1:] * q0barPowNu[1:]
49     return fw
50
51
52
53
54 # Berechne die Änderungsrate Delta x
55 def delta_x(args, t, x):
56     i = np.eye(len(x))
57     dx = la.solve(i - t * jacobi(args, x), fkt_wert(args, x))
```

```

58     return dx
59
60
61
62 # Berechne optimalen Zeitschritt
63 def t_opt(args, t_n, x_n, x_n1, dx):
64     z = abs(np.dot(dx, fkt_wert(args, x_n) - dx))
65     n = 2*la.norm(dx) * la.norm(fkt_wert(args, x_n1) - dx)
66     while z < n:
67         z = 4 * z
68     return t_n * (z / n)
69
70
71 # Berechne SER Zeitschritt
72 def t_ser(args, t_n, x_n, x_n1):
73     z = t_n * la.norm(fkt_wert(args, x_n))
74     n = la.norm(fkt_wert(args, x_n1))
75     return z / n
76
77
78 # Zeige df
79 def plot_df(df):
80     print(df)
81     df.plot()
82     plt.show()
83     return
84
85
86 # Schreibe df als csv in das LaTeX-Verzeichnis
87 def write_df(df, path, file):
88     with open(path + file, 'w') as tf:
89         tf.write(df.to_csv())
90     return
91
92
93 # Berechne Beispiel
94 def calc_bsp(args, x0, t_n, stoffe, max_iter, opt, eps):
95     x_n = x0
96     x_list = [x0]
97     t_list = [t_n]
98     res_list = [la.norm(fkt_wert(args, x0))]
99     i = 0
100    while i in range(0, max_iter) and la.norm(fkt_wert(args, x_n)) > eps:
101        dx = delta_x(args, t_n, x_n)
102        x_n1 = x_n + t_n * dx
103        #Modifizierte Abbuchbedingun, verzichtet auf eine Reduktion des Residuums
104        in den ersten Iterationen
105        if la.norm(fkt_wert(args, x_n1)) >= la.norm(fkt_wert(args, x_n)):
106            if i < 3:
107                res_list = np.vstack((res_list, la.norm(fkt_wert(args, x_n1))))
108                x_list = np.vstack((x_list, x_n1))
109                if opt:
110                    t_n1 = t_opt(args, t_n, x_n, x_n1, dx)
111                else:
112                    t_n1 = t_ser(args, t_n, x_n, x_n1)
113                t_list = np.vstack((t_list, t_n1))
114                x_n = x_n1
115                t_n = t_n1
116            else:

```

```

116         print('Abbruch: Keine Residual-Reduktion')
117         break
118
119     if opt:
120         t_n1 = t_opt(args, t_n, x_n, x_n1, dx)
121     else:
122         t_n1 = t_ser(args, t_n, x_n, x_n1)
123
124     t_list = np.vstack((t_list, t_n1))
125     res_list = np.vstack((res_list, la.norm(fkt_wert(args, x_n1))))
126     x_list = np.vstack((x_list, x_n1))
127     t_n = t_n1
128     x_n = x_n1
129     i = i+1
130     df = pd.DataFrame(data=x_list, columns=stoffe)
131     df['Residuum'] = res_list
132     df['Zeitschrittweite'] = t_list
133     df.index.name = 'Iterationen'
134
135     return df
136
137
138 def main():
139     # Vorgegebene Konstanten für das Problem
140     yCp = np.array([5.8377002519964755e+01,
141                    2.9352296732047269e-03,
142                    1.5061023667222263e-02,
143                    1.3523701213590386e-01])
144     kA = np.array([0.0, 35.5, 1.59, 7.7])
145     kD = np.array([0.0, 1000.0, 1000.0, 1000.0])
146     nu = np.array([0.0, 4.7, 5.29, 3.7])
147     sigma = np.array([0.0, 11.83, 10.6, 10.0])
148     Lambda = 1.2e3
149     args = [yCp, kA, kD, nu, sigma, Lambda]
150
151     # Anfänglicher Zustand
152     x0 = np.array([7e+02, 4e+00, 1.4e+01, 5e+00])
153     # Anfänglich gewählter Zeitschritt
154     t_n = 1
155
156     stoffe = ['$C_a$', '$C_b$', '$C_c$', '$C_d$']
157     max_iter = 50
158     eps = 1e-10
159
160     for opt in [True, False]:
161         path = "../ba_nv_latex/"
162         if opt:
163             print("Beispiel: 2 OPT")
164             file = 'Data_Bsp2.csv'
165             df = calc_bsp(args, x0, t_n, stoffe, max_iter, opt, eps)
166             dfo = df.loc[:, 'Residuum']
167         else:
168             print("\nBeispiel: 2 SER")
169             file = 'Data_Bsp2SER.csv'
170             df = calc_bsp(args, x0, t_n, stoffe, max_iter, opt, eps)
171             dfs = df.loc[:, 'Residuum']
172         plot_df(df)
173         write_df(df, path, file)
174

```

```
175     file = 'Data_Bsp2RES.csv'
176     dfr = pd.merge(dfo, dfs, on='Iterationen', how='outer')
177     dfr.columns = ['OPT', 'SER']
178     plot_df(dfr)
179     write_df(dfr, path, file)
180
181     return
182
183
184 if __name__ == "__main__":
185     main()
```

B.3. Beispiel 3

```
1 import math as m
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import pandas as pd
5 from numpy import linalg as la
6
7
8 # Berechne Jacobi-Matrix
9 def jacobi(args, x, rel_stoffe):
10     k_h = args[0]
11     k_z = args[1]
12     sm = args[2]
13
14     jm = np.zeros((len(rel_stoffe), len(x)))
15
16     jm[0, 0] = -k_z[0] * x[18]
17     jm[0, 1] = k_h[0]
18     jm[0, 18] = -k_z[0] * x[0]
19
20     jm[1, 1] = -k_z[1] * x[18]
21     jm[1, 2] = k_h[1]
22     jm[1, 18] = -k_z[1] * x[1]
23
24     jm[2, 3] = -k_z[2] * x[18]
25     jm[2, 4] = k_h[2]
26     jm[2, 18] = -k_z[2] * x[3]
27
28     jm[3, 5] = -k_z[3] * x[18]
29     jm[3, 6] = k_h[3]
30     jm[3, 18] = -k_z[3] * x[5]
31
32     jm[4, 7] = -k_z[4] * x[18]
33     jm[4, 8] = k_h[4]
34     jm[4, 18] = -k_z[4] * x[7]
35
36     jm[5, 9] = -k_z[5] * x[18]
37     jm[5, 10] = k_h[5]
38     jm[5, 18] = -k_z[5] * x[9]
39
40     jm[6, 11] = -k_z[6] * x[18]
41     jm[6, 12] = k_h[6]
42     jm[6, 18] = -k_z[6] * x[11]
43
44     jm[7, 12] = -k_z[7] * x[18]
45     jm[7, 13] = k_h[7]
46     jm[7, 18] = -k_z[7] * x[12]
47
48     jm[8, 14] = -k_z[8] * x[18]
49     jm[8, 15] = k_h[8]
50     jm[8, 18] = -k_z[8] * x[14]
51
52     jm[9, 16] = -k_z[9] * x[18]
53     jm[9, 17] = k_h[9]
54     jm[9, 18] = -k_z[9] * x[16]
55
56     jm = np.matmul(sm, jm)
57
```

```

58     return jm
59
60
61 # Berechne Funktionswerte
62 def fkt_wert(args, x, rel_stoffe):
63     k_h = args[0]
64     k_z = args[1]
65     sm = args[2]
66
67     fw = np.zeros(len(rel_stoffe))
68
69     fw[0] = k_h[0] * x[1] - k_z[0] * x[0] * x[18]
70     fw[1] = k_h[1] * x[2] - k_z[1] * x[1] * x[18]
71     fw[2] = k_h[2] * x[4] - k_z[2] * x[3] * x[18]
72     fw[3] = k_h[3] * x[6] - k_z[3] * x[5] * x[18]
73     fw[4] = k_h[4] * x[8] - k_z[4] * x[7] * x[18]
74     fw[5] = k_h[5] * x[10] - k_z[5] * x[9] * x[18]
75     fw[6] = k_h[6] * x[12] - k_z[6] * x[11] * x[18]
76     fw[7] = k_h[7] * x[13] - k_z[7] * x[12] * x[18]
77     fw[8] = k_h[8] * x[15] - k_z[8] * x[14] * x[18]
78     fw[9] = k_h[9] * x[17] - k_z[9] * x[16] * x[18]
79
80     fw = np.matmul(sm, fw)
81
82     return fw
83
84 # Berechne Änderungsrate mit Vorkonditionierung
85 def delta_x(args, t, x, rel_stoffe):
86     i = np.eye(len(x))
87     jm = jacobi(args, x, rel_stoffe)
88     am = i - t * jm
89     tm = t_matrix(am)
90
91     ta = np.matmul(tm, am)
92     tat = np.matmul(ta, tm)
93     tat_inv = np.linalg.solve(tat, i)
94     tf = np.matmul(tm, fkt_wert(args, x, rel_stoffe))
95
96     y = np.matmul(tat_inv, tf)
97     dx = np.matmul(tm, y)
98
99     return dx
100
101
102 # Berechne optimalen Zeitschritt
103 def t_opt(args, t_n, x_n, x_n1, dx, rel_stoffe):
104     z = abs(np.matmul(np.transpose(dx), fkt_wert(args, x_n, rel_stoffe) - dx))
105     n = 2 * la.norm(dx) * la.norm(fkt_wert(args, x_n1, rel_stoffe) - dx)
106     return (z / n) * t_n
107
108 # Berechne SER Zeitschritt
109 def t_ser(args, t_n, x_n, x_n1, rel_stoffe):
110     z = t_n * la.norm(fkt_wert(args, x_n, rel_stoffe))
111     n = la.norm(fkt_wert(args, x_n1, rel_stoffe))
112     return z / n
113
114 # Berechne D Matrix zur Vorkonditionierung
115 def d_matrix(am):
116     dm = np.zeros((len(am), len(am)))

```



```

117     for i in range(0, len(am)):
118         dm[i, i] = 1 / max(abs(am[i, :]))
119     return dm
120
121
122 # Berechne T Matrix zur Vorkonditionierung
123 def t_matrix(am):
124     tm = np.zeros((len(am), len(am)))
125     for i in range(0, len(am)):
126         tm[i, i] = m.sqrt(1 / max(abs(am[i, :]))))
127     return tm
128
129
130 # Zeige df
131 def plot_df(df, stoffe):
132     pd.set_option('display.max_columns', 19)
133     print(df)
134
135     df.plot()
136     plt.show()
137     plt.xlabel('Iterationen')
138     plt.ylabel('Konzentrationen')
139     plt.legend(stoffe)
140
141     return
142
143
144 # Schreibe df als csv in das LaTeX-Verzeichnis
145 def write_df(df, path, file):
146     # Schreibe Gesamttabelle
147     with open(path + file, 'w') as tf:
148         tf.write(df.to_csv())
149
150     # Erzeuge drei Teiltabellen zur besseren Darstellung in LaTeX
151     a = ['$Dap$', '$Dap+$', '$Dap2+$', '$Pip$', '$Pip+$', '$Pip2+$']
152     b = ['$Dea$', '$Dea+$', '$Tris$', '$Tris+$', '$Imi$', '$Imi+$']
153     c = ['$Bts$', '$Bts+$', '$Ace-$', '$Ace$', '$Lac-$', '$Lac$']
154     d = ['Residuum', 'Zeitschrittweite']
155
156     dfa = df.loc[:, a]
157     dfb = df.loc[:, b]
158     dfc = df.loc[:, c]
159     dfd = df.loc[:, d]
160
161     with open(path + file[:-4] + 'a.csv', 'w') as tf:
162         tf.write(dfa.to_csv())
163     with open(path + file[:-4] + 'b.csv', 'w') as tf:
164         tf.write(dfb.to_csv())
165     with open(path + file[:-4] + 'c.csv', 'w') as tf:
166         tf.write(dfc.to_csv())
167     with open(path + file[:-4] + 'd.csv', 'w') as tf:
168         tf.write(dfd.to_csv())
169
170     return
171
172
173 # Berechne Beispiel
174 def calc_bsp(args, x0, t_n, stoffe, rel_stoffe, max_iter, opt, eps):
175     x_n = x0

```

```

176 x_list = [x0]
177 t_list = [t_n]
178 res_list = [la.norm(fkt_wert(args, x_n, rel_stoffe))]
179 i= 0
180 while i in range(0, max_iter) and la.norm(fkt_wert(args, x_n, rel_stoffe)) >
eps:
181     dx = delta_x(args, t_n, x_n, rel_stoffe)
182     x_n1 = x_n + t_n * dx
183
184     if la.norm(fkt_wert(args, x_n1, rel_stoffe)) > la.norm(fkt_wert(args, x_n,
rel_stoffe))):
185         print('Abbruch: Keine Residual-Reduktion')
186         break
187     if opt:
188         t_n1 = t_opt(args, t_n, x_n, x_n1, dx, rel_stoffe)
189     else:
190         t_n1 = t_ser(args, t_n, x_n, x_n1, rel_stoffe)
191     if t_n1 < t_n:
192         print('Abbruch: Maximaler Zeitschritt erreicht')
193         break
194     t_list = np.vstack((t_list, t_n1))
195     res_list = np.vstack((res_list, la.norm(fkt_wert(args, x_n1, rel_stoffe))))
196     x_list = np.vstack((x_list, x_n1))
197     x_n = x_n1
198     t_n = t_n1
199     i = i + 1
200
201 df = pd.DataFrame(data=x_list, columns=stoffe)
202 df['Residuum'] = res_list
203 df['Zeitschrittweite'] = t_list
204 df.index.name = 'Iterationen'
205
206 return df
207
208
209 def main():
210     # Vorgegebene Konstanten für das Problem
211     k_h = np.array([2.4e-8, 2.29e-6, 1.31e-6, 8.47e-6, 1.02e-4,
212                     3.28e-4, 1.86e-7, 4.6e-3, 1.75e-2, 1.38e-1])
213     k_z = np.array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
214     sm = np.array([[1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
215                   [-1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
216                   [0, -1, 0, 0, 0, 0, 0, 0, 0, 0],
217                   [0, 0, 1, 0, 0, 0, 0, 0, 0, 0],
218                   [0, 0, -1, 0, 0, 0, 0, 0, 0, 0],
219                   [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
220                   [0, 0, 0, -1, 0, 0, 0, 0, 0, 0],
221                   [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
222                   [0, 0, 0, 0, -1, 0, 0, 0, 0, 0],
223                   [0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
224                   [0, 0, 0, 0, 0, -1, 0, 0, 0, 0],
225                   [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
226                   [0, 0, 0, 0, 0, 0, -1, 1, 0, 0],
227                   [0, 0, 0, 0, 0, 0, 0, -1, 0, 0],
228                   [0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
229                   [0, 0, 0, 0, 0, 0, 0, 0, -1, 0],
230                   [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
231                   [0, 0, 0, 0, 0, 0, 0, 0, 0, -1],
232                   [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]])

```

```

233     args = [k_h, k_z, sm]
234
235     # Anfängliche Konzentrationen
236     x0 = np.array([10, 0, 0, 10, 0,
237                   10, 0, 10, 0, 10,
238                   0, 10, 0, 0, 0,
239                   10, 0, 10, 10 ** -0.5])
240
241     # Anfänglich gewählter Zeitschritt
242     t_n = 1
243
244     stoffe = [ '$Dap$', '$Dap+$', '$Dap2+$', '$Dea$', '$Dea+$',
245              '$Tris$', '$Tris+$', '$Imi$', '$Imi+$', '$Bts$',
246              '$Bts+$', '$Pip$', '$Pip+$', '$Pip2+$',
247              '$Ace-$', '$Ace$', '$Lac-$', '$Lac$', '$H+$' ]
248     rel_stoffe = [ 'Dap+', 'Dap2+', 'Dea+', 'Tris+', 'Imi+',
249                  'Bts+', 'Pip+', 'Pip2+', 'Ace', 'Lac' ]
250
251     max_iter = 50
252     eps = 1e-10
253     for opt in [True, False]:
254         path = "../ba_nv_latex/"
255         if opt:
256             print("Beispiel: 3 OPT")
257             file = 'Data_Bsp3.csv'
258             df = calc_bsp(args, x0, t_n, stoffe, rel_stoffe, max_iter, opt, eps)
259             dfo = df.loc[:, 'Residuum']
260         else:
261             print("\nBeispiel: 3 SER")
262             file = 'Data_Bsp3SER.csv'
263             df = calc_bsp(args, x0, t_n, stoffe, rel_stoffe, max_iter, opt, eps)
264             dfs = df.loc[:, 'Residuum']
265             plot_df(df, stoffe)
266             write_df(df, path, file)
267
268 if __name__ == "__main__":
269     main()

```

B.4. Vergleich der Robustheit

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import pandas as pd
4
5 # Importiere zu testendes Verfahren und Vergleichsverfahren
6 import Bsp2_Rob
7 import SmaTest_Rob
8
9
10 # Erzeuge Sphäre mit gleichverteilten Punkten
11 def points(nDim, nSamples, r_min, r_max):
12     # Richtung ziehen
13     dirs = np.random.normal(size=(nDim, nSamples))
14     norms = np.linalg.norm(dirs, axis=0)
15     dirs = dirs / norms
16
17     # Radius ziehen
18     rad = np.power(np.random.uniform(
19         low=r_min ** nDim, high=r_max ** nDim,
20         size=(nSamples,)), 1.0 / nDim)
21
22     return dirs * rad
23
24
25 # Zeige df
26 def plot_df(df):
27     #print(df)
28     df.plot()
29     plt.show()
30     return
31
32
33 # Schreibe df als csv in das LaTeX-Verzeichnis
34 def write_df(df, path, file):
35     with open(path + file, 'w') as tf:
36         tf.write(df.to_csv())
37     return
38
39
40 # Berechne Robustheit
41 def calc_rob(args, liste1, liste2):
42     nSamples = args[0]
43     nMin = args[1]
44     nDim = args[2]
45     r_min = args[3]
46     r_max = args[4]
47     normmin = args[5]
48     stepsize = args[6]
49     maxDistance = args[7]
50     x_ref = args[8]
51
52     df_g = pd.DataFrame(columns=liste1)
53     df_t = pd.DataFrame(columns=liste2)
54
55     # Vergrößert Abstand von x_ref
56     for step in range(0, maxDistance):
57         # Zähle wie oft die Verfahren konvergieren
```

```

58     counterPTC = 0
59     counterNEW = 0
60     print(step)
61     # Zähle legale Punkte
62     nCounter = nSamples
63
64     # Vergrößere Abstand
65     r_min = r_min + stepsize
66     r_max = r_max + stepsize
67
68     # Erzeuge Punkte und transformiere in eine Ellipse
69     sphToEllips = np.diag([1000, 10, 10, 10])
70     pts = points(nDim, nSamples, r_min, r_max)
71     pts = np.dot(sphToEllips, pts)
72
73     # Für alle erzeugten Punkte ...
74     for i in range(0, nSamples):
75         x = []
76         skip = False
77
78         # Für jeden Punkt gehe von x_ref aus und erhalte neuen Punkt
79         for j in range(0, nDim):
80             x_j = x_ref[j] + pts[j, i]
81             if x_j < 0:
82                 skip = True
83             x.append(x_j)
84
85         # Entferne negative Punkte
86         if skip:
87             nCounter = nCounter - 1
88         else:
89             # Ruft Bsp2-Verfahren auf und gibt Residual Norm zurück
90             norm1 = Bsp2_Rob.main(x)
91             if norm1 < normmin:
92                 counterPTC = counterPTC + 1
93
94             # Ruft Vergleichsverfahren auf und gibt Residual Norm zurück
95             try:
96                 norm2 = SmaTest_Rob.smaTestRob(x)
97             except ValueError:
98                 norm2 = float('nan')
99
100             if norm2 < normmin:
101                 counterNEW = counterNEW + 1
102
103             if np.isnan(norm1) and np.isnan(norm2):
104                 nCounter = nCounter - 1
105
106     if nCounter < nMin:
107         print("Abbruch: Nicht mehr genügend legale Punkte")
108         break
109
110     # Berechne Anzahl der konvergierten legalen Punkte
111     wert1 = counterPTC / nCounter
112     wert2 = counterNEW / nCounter
113
114     # Erzeuge df mit Prozent der konvergierten Punkte
115     # ... für Ausgabe als Plot
116     df_g.loc[step] = [wert1, wert2]

```

```

117         # ... und für Ausgabe als Tabelle
118         df_t.loc[step] = [wert1, wert2, nCounter, r_min]
119         df_t.index.name = 'Iterationen'
120
121     return df_g, df_t
122
123
124 def main():
125     # Parameter
126     nSamples = 100000 # Typical: 100000
127     nMin = nSamples / 100
128     nDim = 4
129     r_min = 0
130     r_max = 0.1
131     normmin = 0.02
132     stepsize = 0.1
133     maxDistance = 301
134     x_ref = np.array([1.0485785488181000e+03,
135                      1.1604726694141368e+01,
136                      1.1469542586742687e+01,
137                      9.7852311988018670e+00])
138     args = [nSamples, nMin, nDim, r_min, r_max, normmin, stepsize, maxDistance,
139            x_ref]
140
141     liste1 = ['Bsp2', 'smaTest']
142     liste2 = ['Bsp2', 'smaTest', 'Punkte', 'rmin']
143
144     path = "../ba_nv_latex/"
145     file = 'Data_Robustheit.csv'
146
147     df_g, df_t = calc_rob(args, liste1, liste2)
148
149     plot_df(df_g)
150     write_df(df_t, path, file)
151
152     return
153
154 if __name__ == "__main__":
155     main()

```

Eigenständigkeitserklärung

Hiermit versichere ich, Neil Vetter, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, dass alle Stellen der Arbeit, die wörtlich oder sinngemäß aus anderen Quellen übernommen wurden, als solche kenntlich gemacht worden sind, und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt wurde.

Köln, den 21. November 2019

Neil Vetter